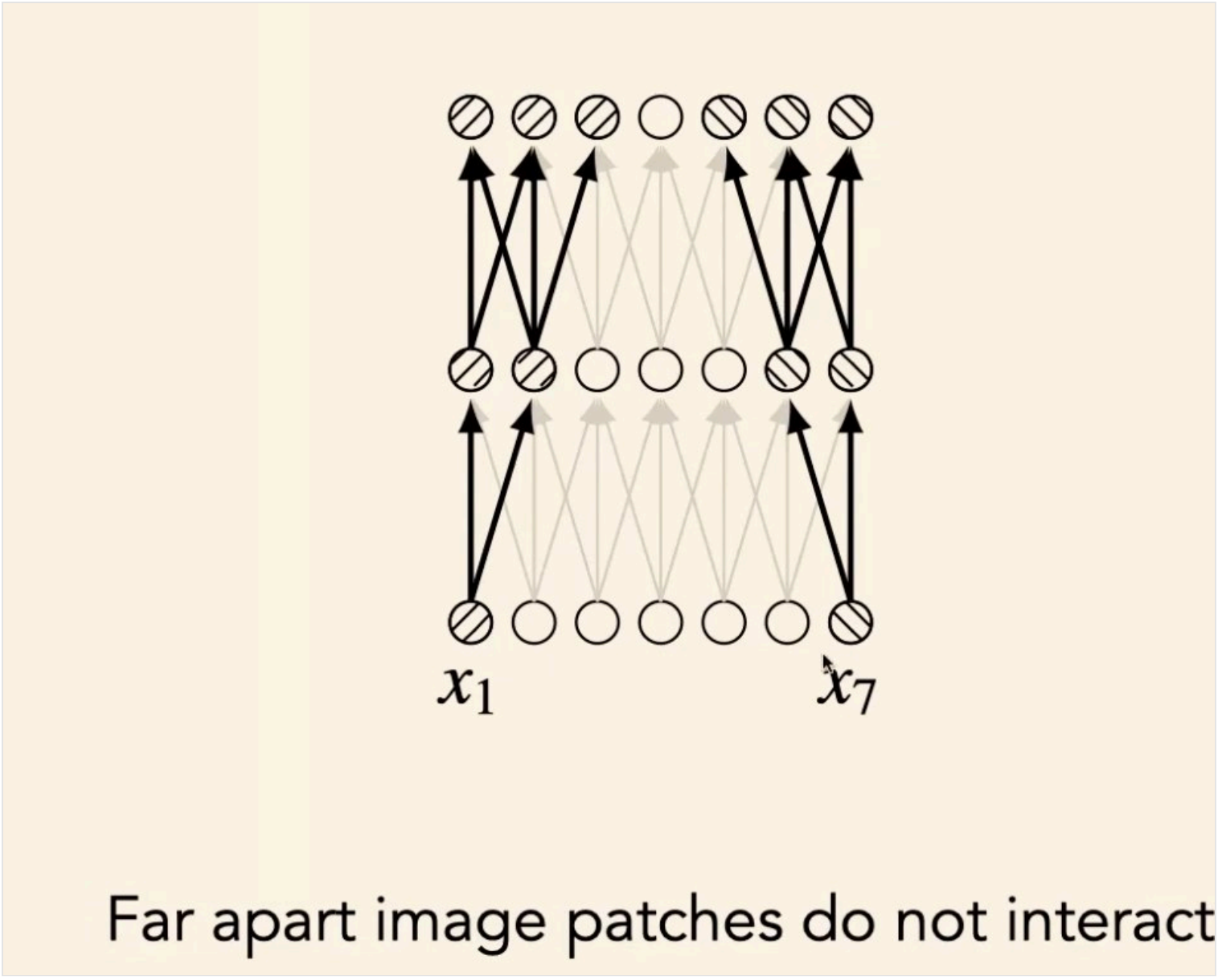
Lec. transformer

- CNN의 단점

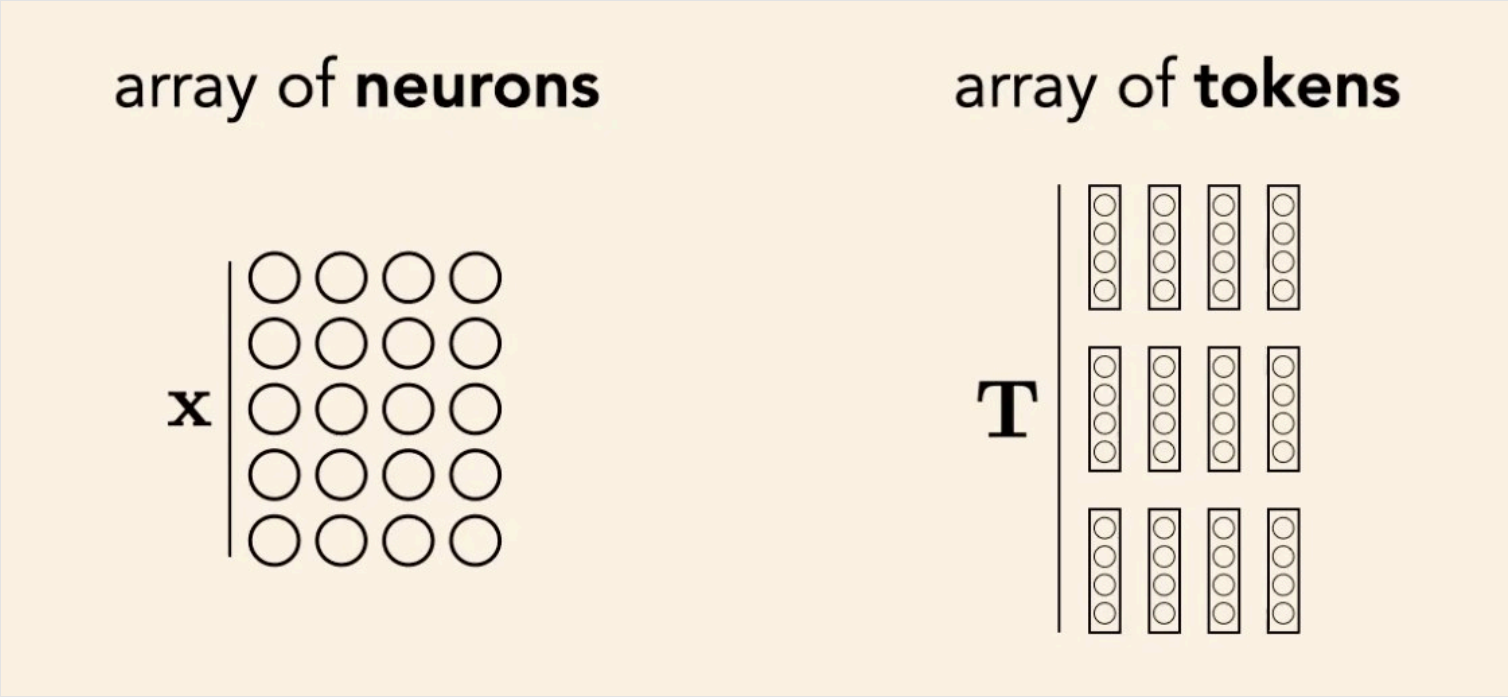


- 단일 convolution layer에서는 receptive field(수용영역) 밖에 있는 멀리 떨어진 패치가 해당 출력에 직접 영향을 주지 못함.
- 다만 여러 convolution layer를 쌓거나, 큰 kernel 또는 dilated convolution을 사용하여 receptive field를 확장하면 멀리 있는 패치의 정보도 출력에 영향을 줄 수 있음.
- 멀리 떨어진 패치 사이의 정보를 전달하려면 convolution layer를 여러 개 쌓거나 receptive field를 넓혀야 하므로 추가 계산이 필요할 수 있음.
- 이는 모든 토큰 사이의 관계를 한 층에서 직접 계산할 수 있는 global attention과 비교한 것이며, CNN이 항상 계산량이 더 많다는 뜻은 아님. Global attention도 토큰 수가 증가하면 계산량이 크게 늘어남.
- → attention을 써서, 문제와 관련된 것에 집중

1 tokens

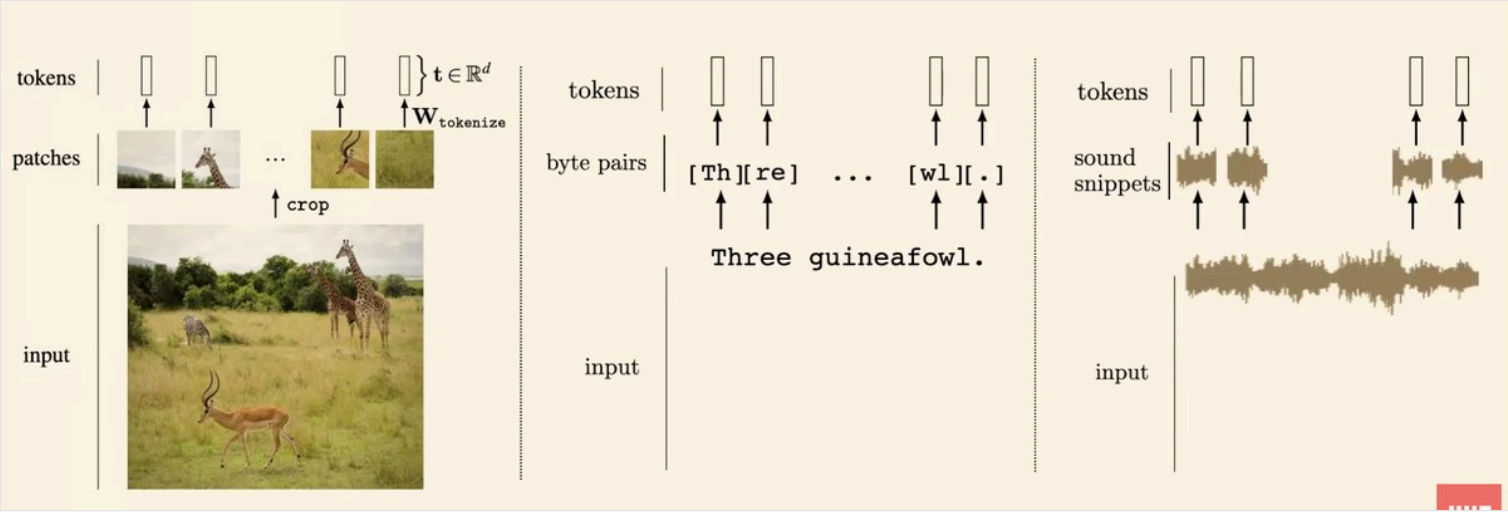
- token = 입력 데이터를 나눈 기본 단위.

- (NLP에서는 단어, subword, 문자 등이 token이 될 수 있고, 이미지에서는 patch를 token으로 사용할 수 있음.)
- token embedding(token vector) = 각 token을 신경망이 처리할 수 있도록 표현한 벡터.
- 즉, token은 입력의 단위이고, embedding은 해당 token을 표현하는 벡터이므로 둘을 구분해야 함.
- array of neurons → array of tokens로 생각해야 함.
- 여기서 array of tokens는 각 token에 대응하는 embedding 벡터들의 배열로 이해할 수 있음.
- 신경망 - 스칼라 값을 갖는 뉴런들이 있음.
- 토큰 신경망 - 각 token을 벡터로 표현하여, 벡터 값을 갖는 유닛들로 구성되어 있음.



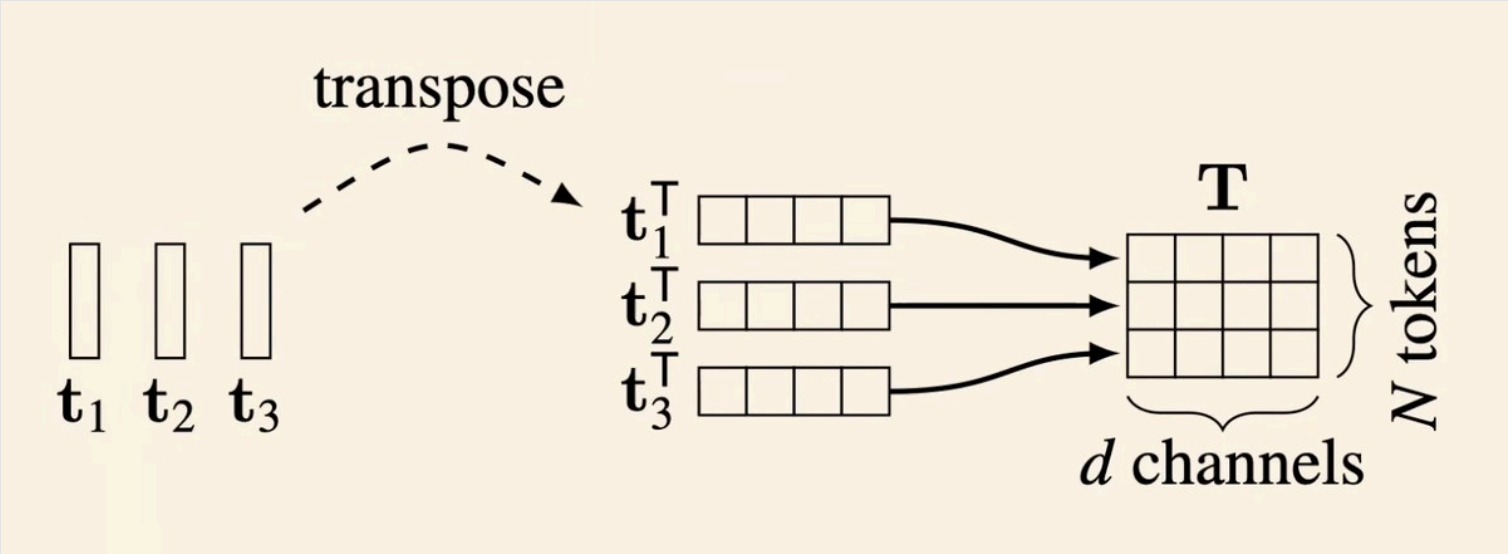
- 여기서도 컨볼루션 같은 걸 할 수 있음

- Tokenizing the input data
- data → token으로 바꿔야 함
- for image. 패치로 분할 → W로 tokenize



- input → byte pairs → tokens 순으로 토큰화

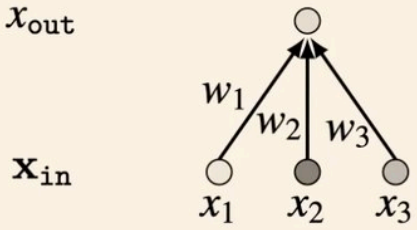
- Notation



- token들을 →matrix T로 만듦.

- Linear combination of tokens

Linear combination of neurons

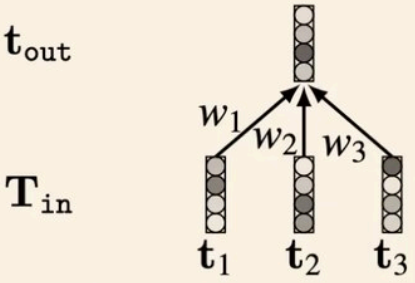


$$x_{\text{out}} = w_1 x_1 + w_2 x_2 + w_3 x_3$$

$$x_{\text{out}}[i] = \sum_{j=1}^{N} w_{ij} x_{\text{in}}[j]$$

$$\mathbf{x}_{\text{out}} = \mathbf{W}\mathbf{x}_{\text{in}}$$

Linear combination of tokens



$$\mathbf{t}_{\text{out}} = w_1 \mathbf{t}_1 + w_2 \mathbf{t}_2 + w_3 \mathbf{t}_3$$

$$\mathbf{T}_{\text{out}}[i,:] = \sum_{j=1}^{N} w_{ij} \mathbf{T}_{\text{in}}[j,:]$$

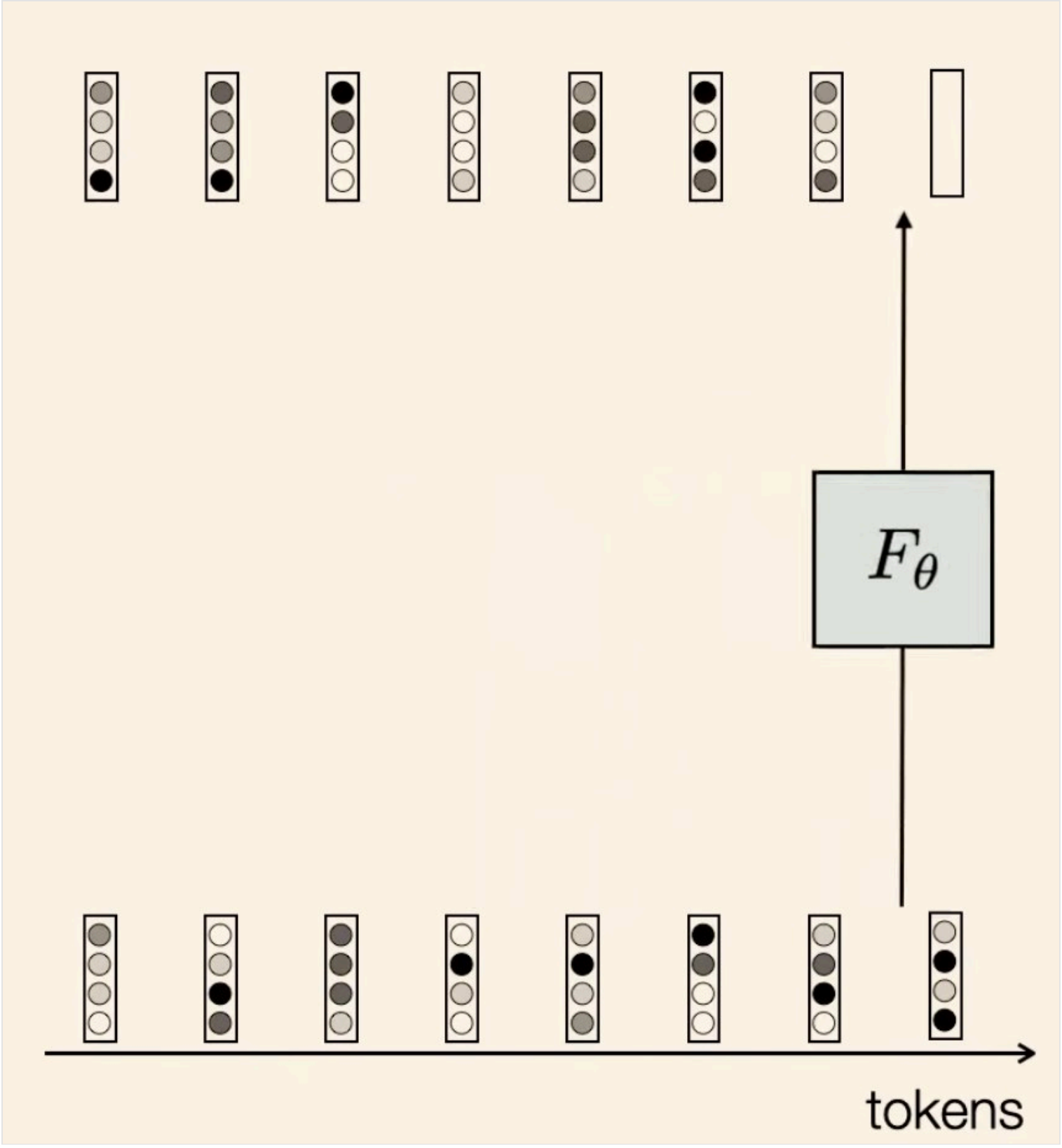
$$\mathbf{T}_{\text{out}} = \mathbf{W}\mathbf{T}_{\text{in}}$$

- Token-wise nonlinearity

$$\mathbf{x}_{\text{out}} = \begin{bmatrix} \texttt{relu}(x_{\text{in}}[0]) \\ \vdots \\ \texttt{relu}(x_{\text{in}}[N-1]) \end{bmatrix}$$

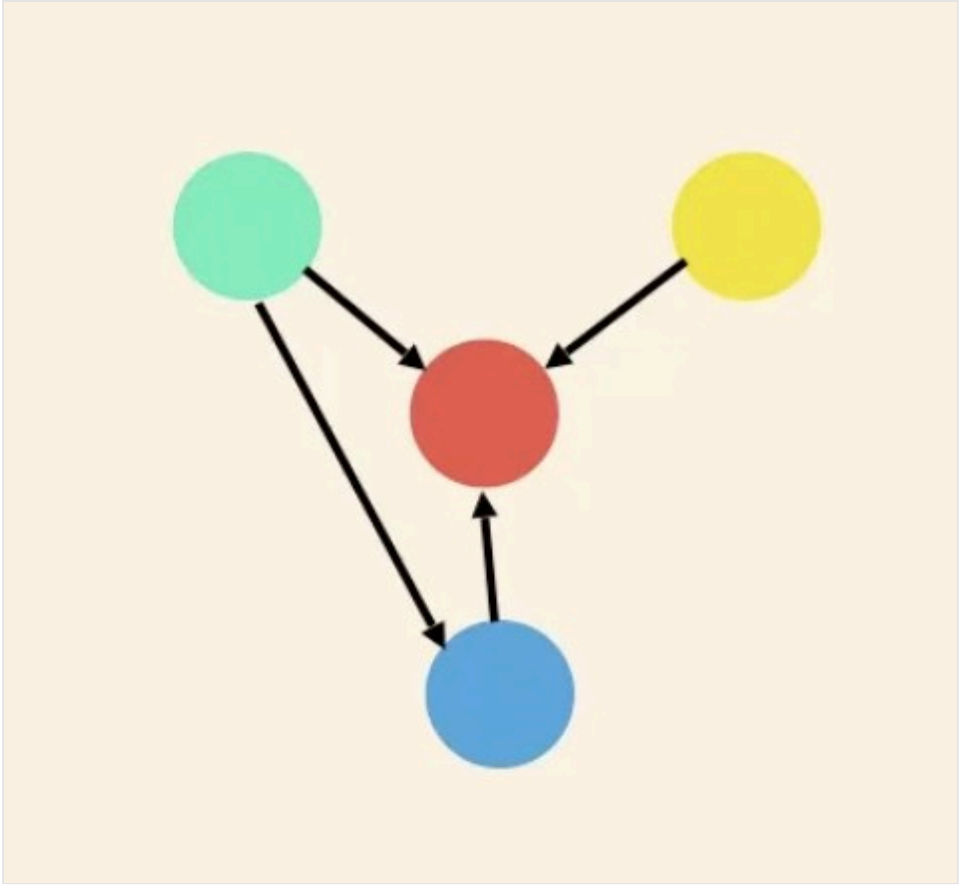
$$\mathbf{T}_{\text{out}} = \begin{bmatrix} F_\theta(\mathbf{T}_{\text{in}}[0,:]) \\ \vdots \\ F_\theta(\mathbf{T}_{\text{in}}[N-1,:]) \end{bmatrix}$$

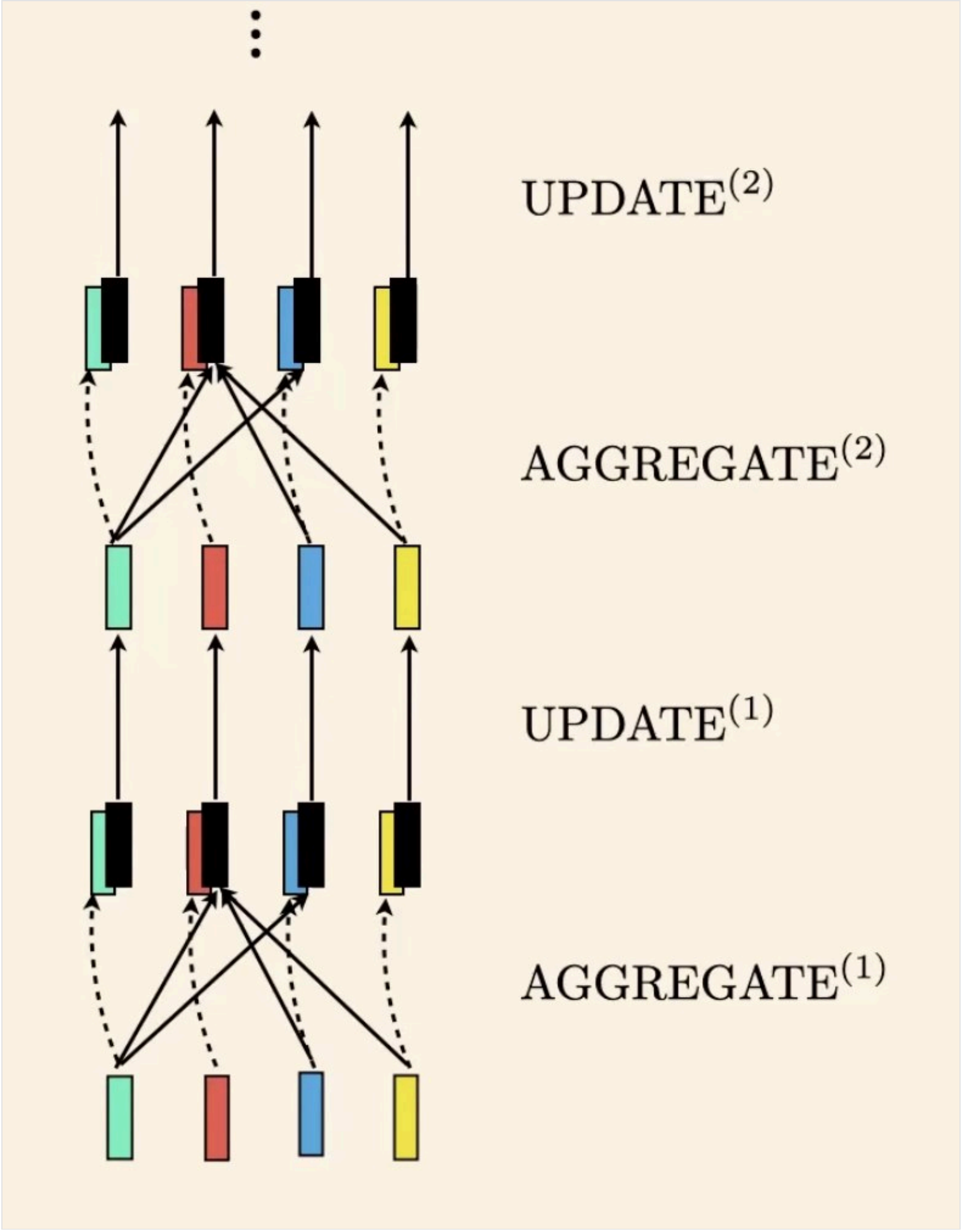
- F를 1x1 컨볼루션 처럼 보아도 됨.



- ReLU처럼 nonlinearity를 만들던 역할을 token에서는 F가 대신함.

- GNN
- 화살표 방향에 있는 거에 영향을 주며 update 되는 구조임

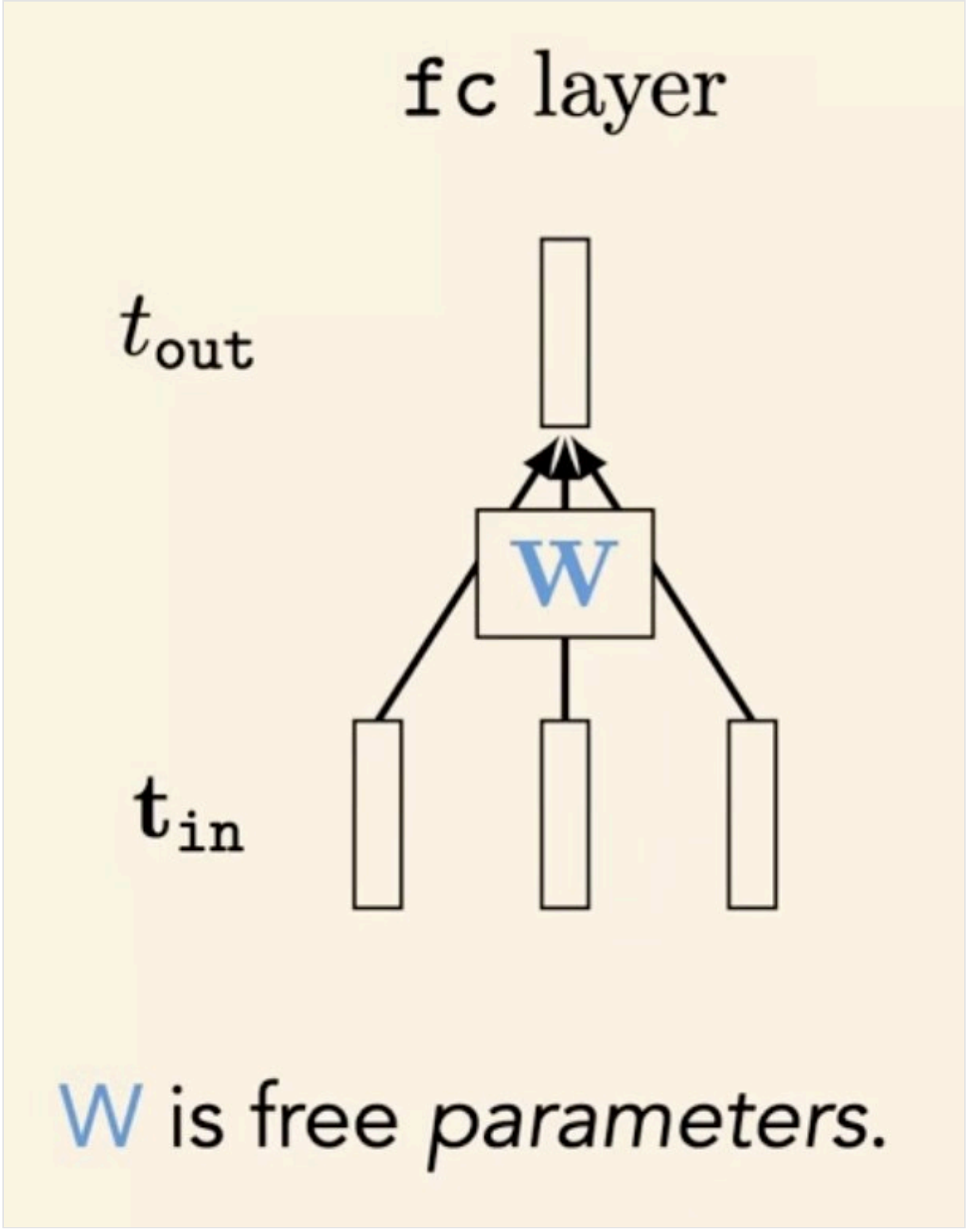




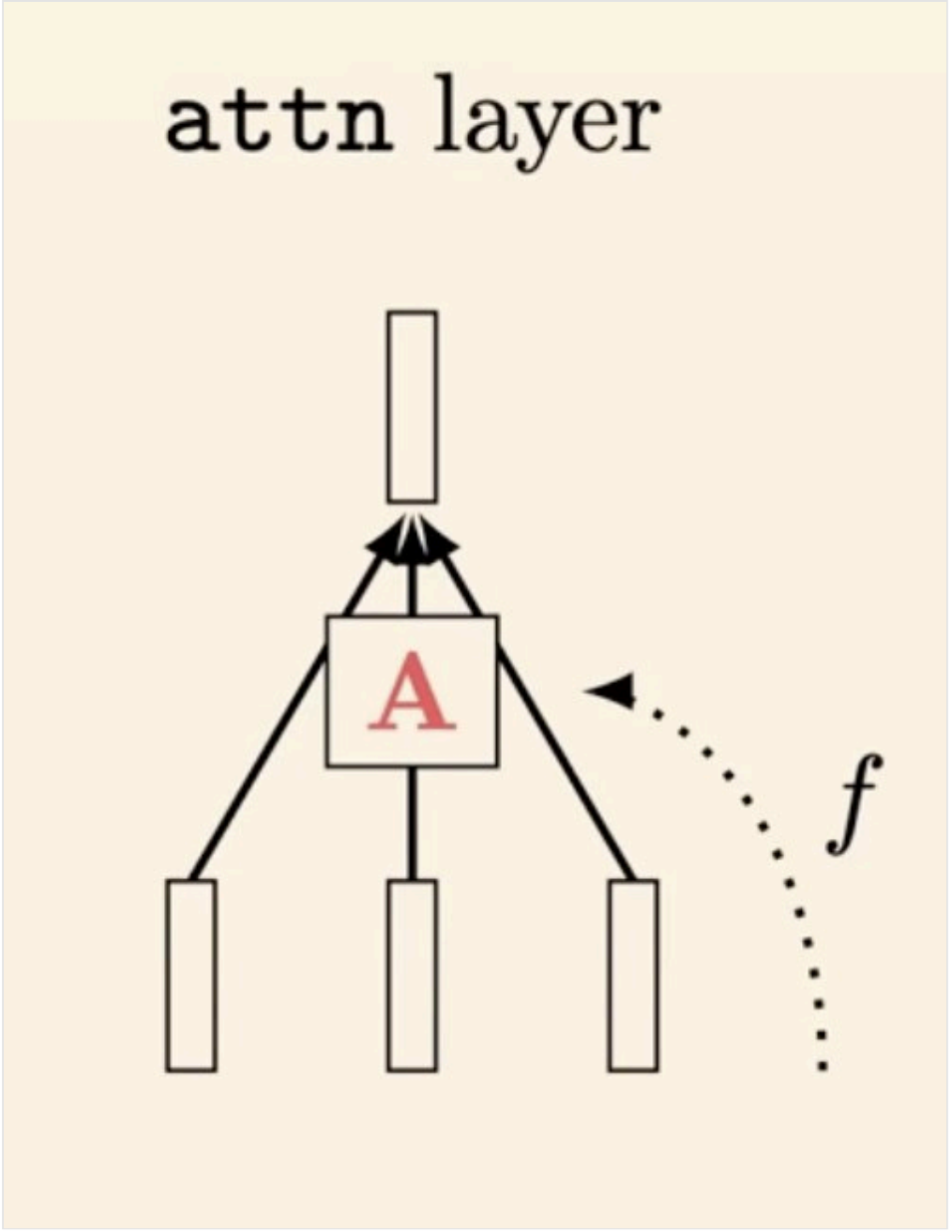
- 트랜스포머는 fully-connected graphs로 볼 수 있음

---

2 attention



- 원래 토큰이 학습되는 방식



- Attention에서 A(attention weight matrix) 자체는 독립적으로 학습되는 자유 파라미터(free parameter)가 아니라, 입력 데이터에 따라 동적으로 계산되는 값이다.
- 구체적으로 query와 key로부터 attention score를 계산하고, softmax를 적용하여 A를 얻는다. 따라서 입력이 달라지면 A도 달라진다.
- 그러나 attention 메커니즘 전체에 학습 파라미터가 없는 것은 아니다. Query, key, value를 생성하는 projection 행렬 W_q, W_k, W_v와 최종 output projection 행렬 W_o는 학습 가능한 파라미터이다.
- 즉, A는 입력으로부터 계산되는 attention weight이고, W_q, W_k, W_v, W_o는 학습 과정에서 업데이트되는 파라미터이다.

$$A = f(\dots)$$
$$T_{out} = AT_{in}$$

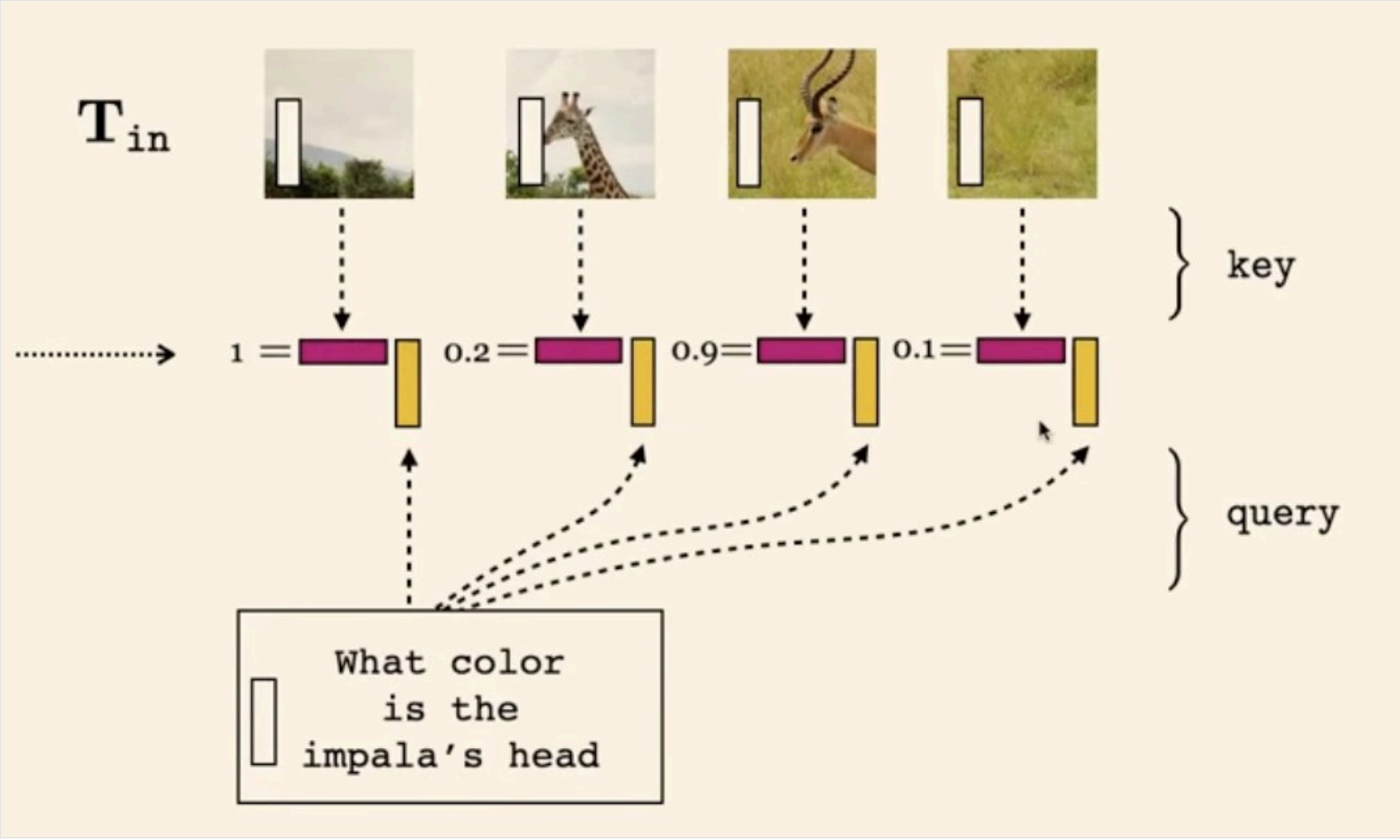
- 직관적으로
  ex1) How many animals are in the photo?
  attend → animals의 head.
  ex2) What is the color of the impala?
  attend → impala

- 이런 식으로 중요한 정보에 attend해야 한다.

---

query-key-value attention

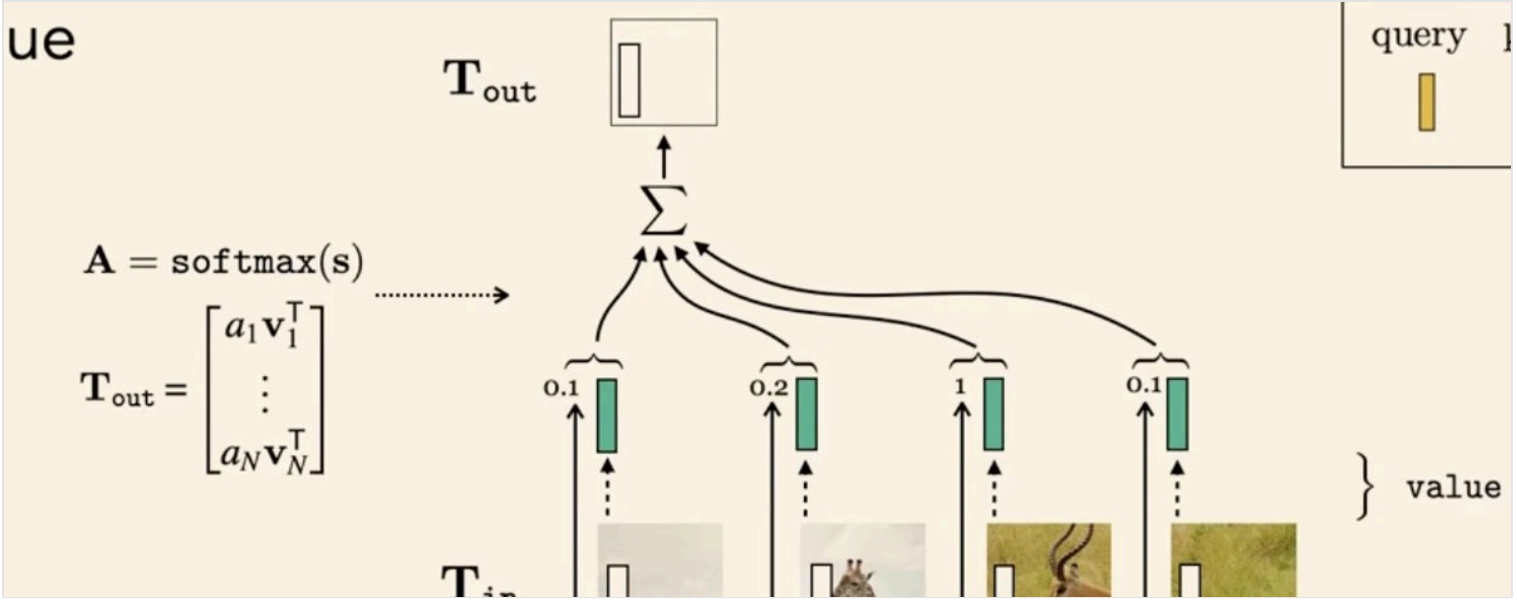
- question → attention을 판단함.
- Query : $q = W_q t$
  Key : $k = W_k t$



- (이미지라면) 각 patch들이 key를 가지고 있고,
  이곳에다가 query를 곱함으로서(유사성을 확인해봄)
  → 이게 attend할만한 것인지를 판단하는 거임

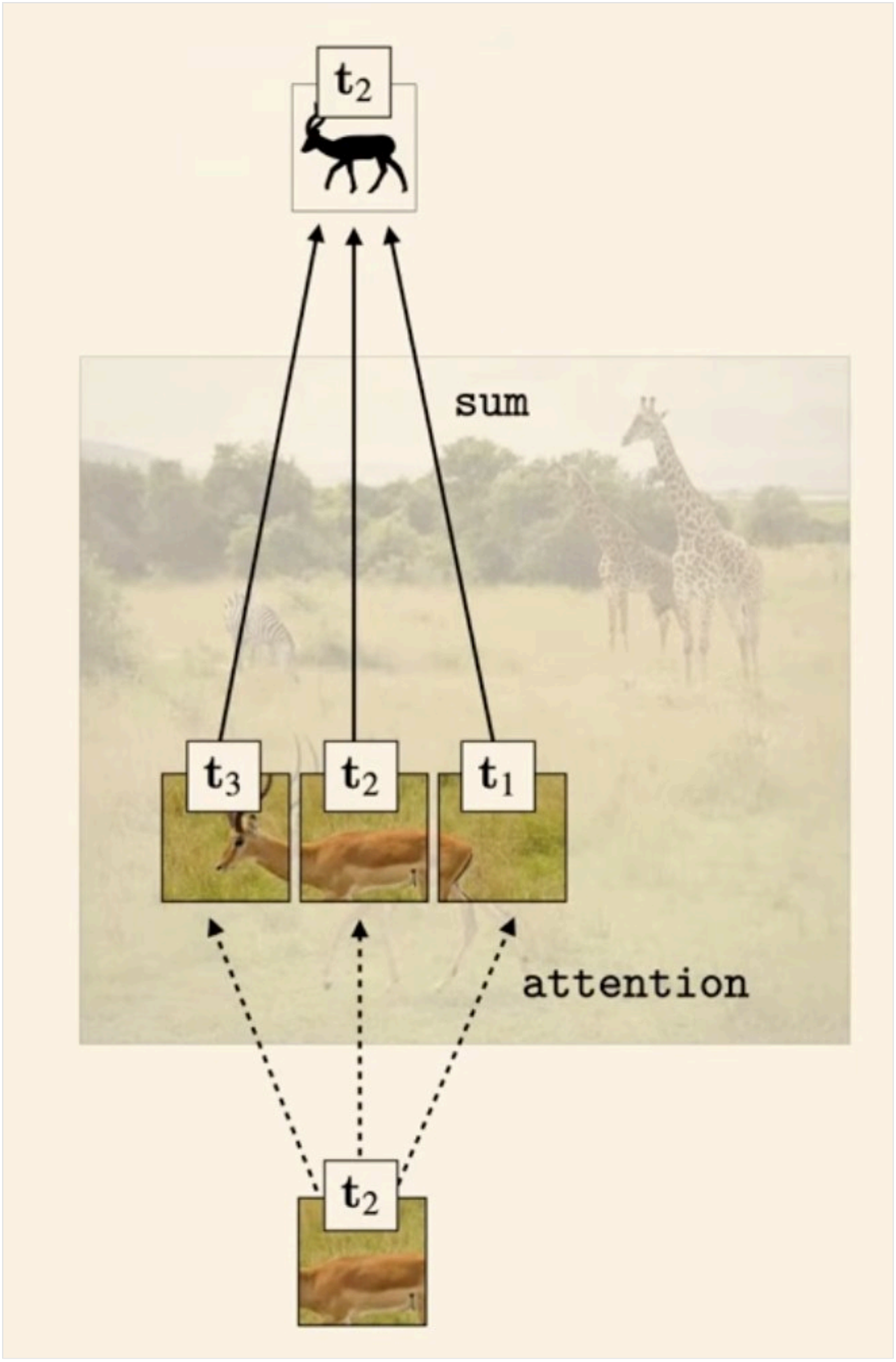
$$\mathbf{s} = [\mathbf{q}_{\text{question}}^T \mathbf{k}_1, \ldots, \mathbf{q}_{\text{question}}^T \mathbf{k}_N]$$

- 이런 내적값(s : 유사율)이 됨.
- Value : $v = W_v t$



- Attention은 query와 key 사이의 유사도를 계산하여 각 value에 부여할 가중치를 구하는 방식이다.
- Scaled dot-product attention에서는 query Q와 key K의 내적을 이용하여 attention score s를 계산한다.
- $s = QK^T / \sqrt{d_k}$
- $A = \text{softmax}(s) = \text{softmax}(QK^T / \sqrt{d_k})$
- 여기서 d_k는 key 벡터의 차원이다. 내적의 크기는 차원이 커질수록 커질 수 있으므로 $\sqrt{d_k}$로 나누어 score의 크기를 조정한다. 이 scaling이 없으면 큰 내적값으로 인해 softmax가 포화되어 기울기가 매우 작아질 수 있다.
- Softmax는 각 query에 해당하는 행(row)마다 key 방향으로 적용되며, 각 행의 attention weight 합은 1이 된다.
- 이렇게 계산된 attention weight A를 value V에 곱하여 출력값을 얻는다.
- output = AV
- 따라서 A는 query와 key의 유사도로부터 계산된 가중치 행렬이며, 실제 attention 출력은 이 가중치를 value에 적용한 결과이다.
- A(attention) · value → T_{out}
- Attention에서는 attention weight matrix A를 value matrix V에 곱하여 출력을 계산한다.
- 올바른 행렬 곱 순서는 A·V이며, V·A가 아니다.
- Self-attention에서 토큰 수가 N이고 value 벡터의 차원이 d_v이면 각 행렬의 크기는 다음과 같다.
- • A: $N \times N$
- • V: $N \times d_v$
- • AV: $N \times d_v$
- 따라서 attention의 출력은 다음과 같다.
- $T_{out} = A\ V_{in}$
- 여기서 A는 각 query 토큰이 key 토큰들에 부여하는 attention weight이고, V_{in}은 입력 value matrix이다.
- A와 V_{in}을 곱하면 각 query 위치에서 value 벡터들의 가중합이 계산되며, 결과 T_{out}의 크기는 $N \times d_v$가 된다.
- 즉, attention 출력은 value에 attention을 뒤에서 곱하는 것이 아니라, attention weight A를 왼쪽에서 value V에 곱하여 계산한다.

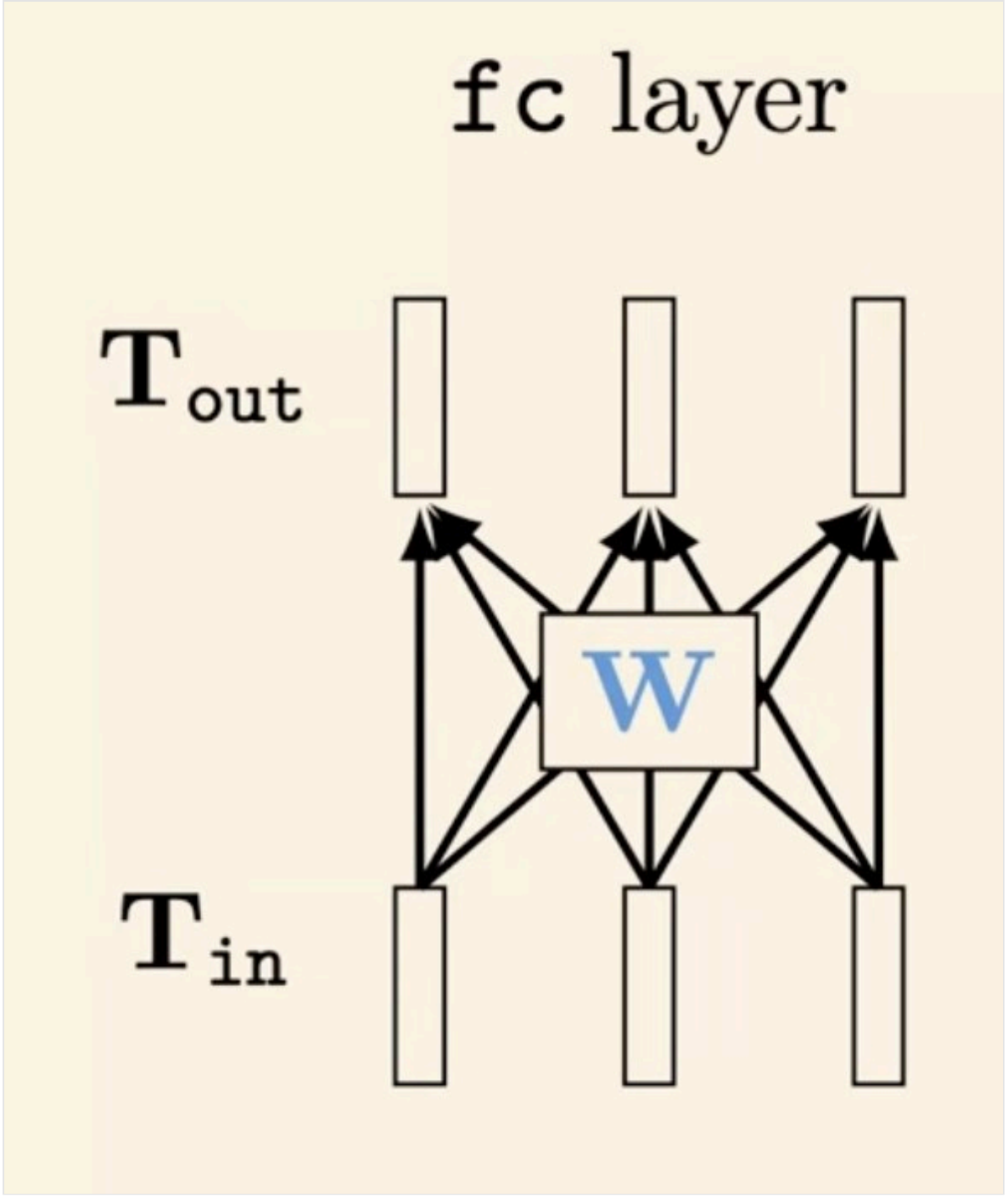
self attention

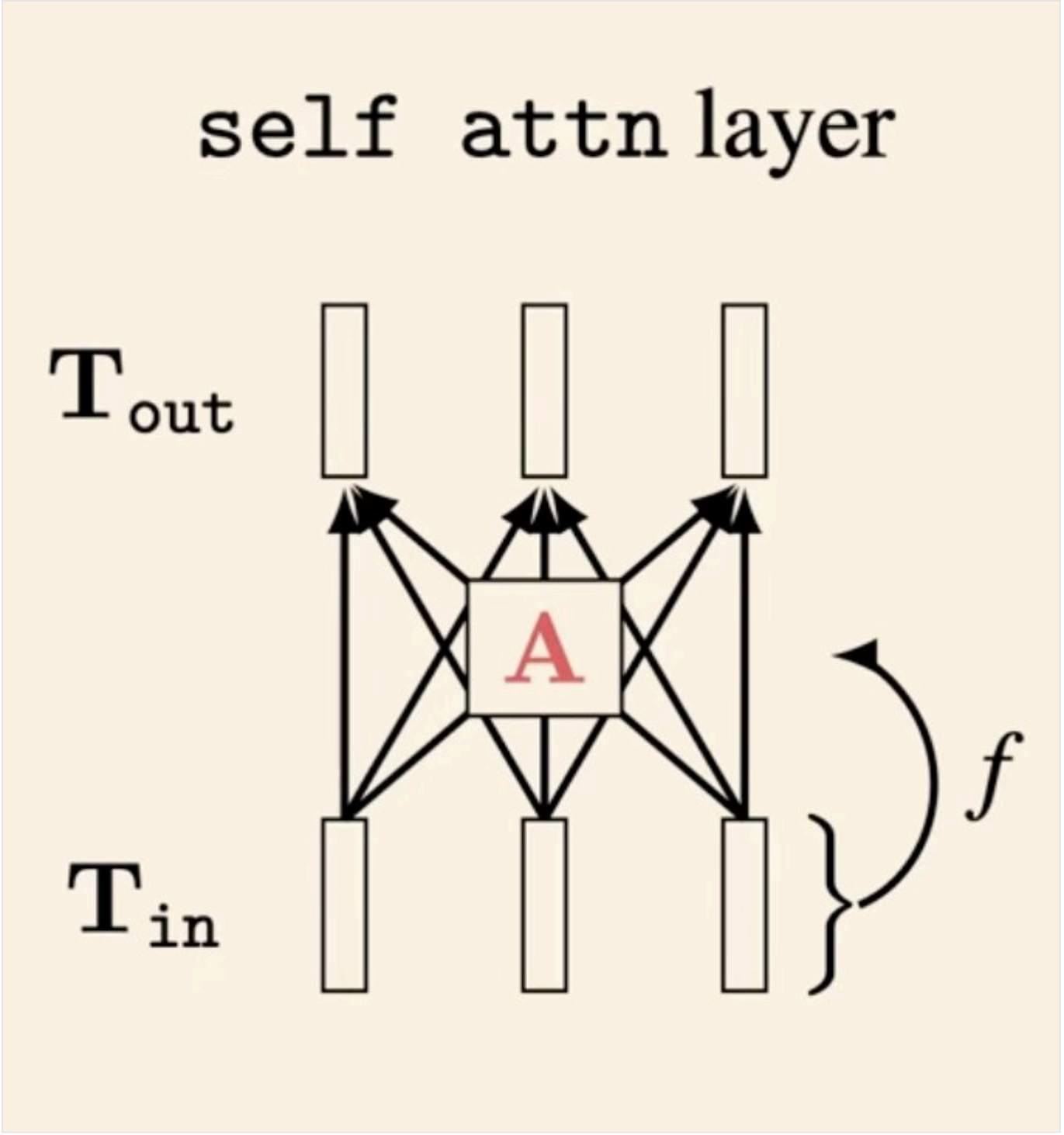


- 외부 question으로부터 Attention하는 게 아니라,
  data 자체가 스스로 attention하는 거임.
- ex) t2는 혼자 있는 것보다, t3,t1이랑 유사도가 있음.

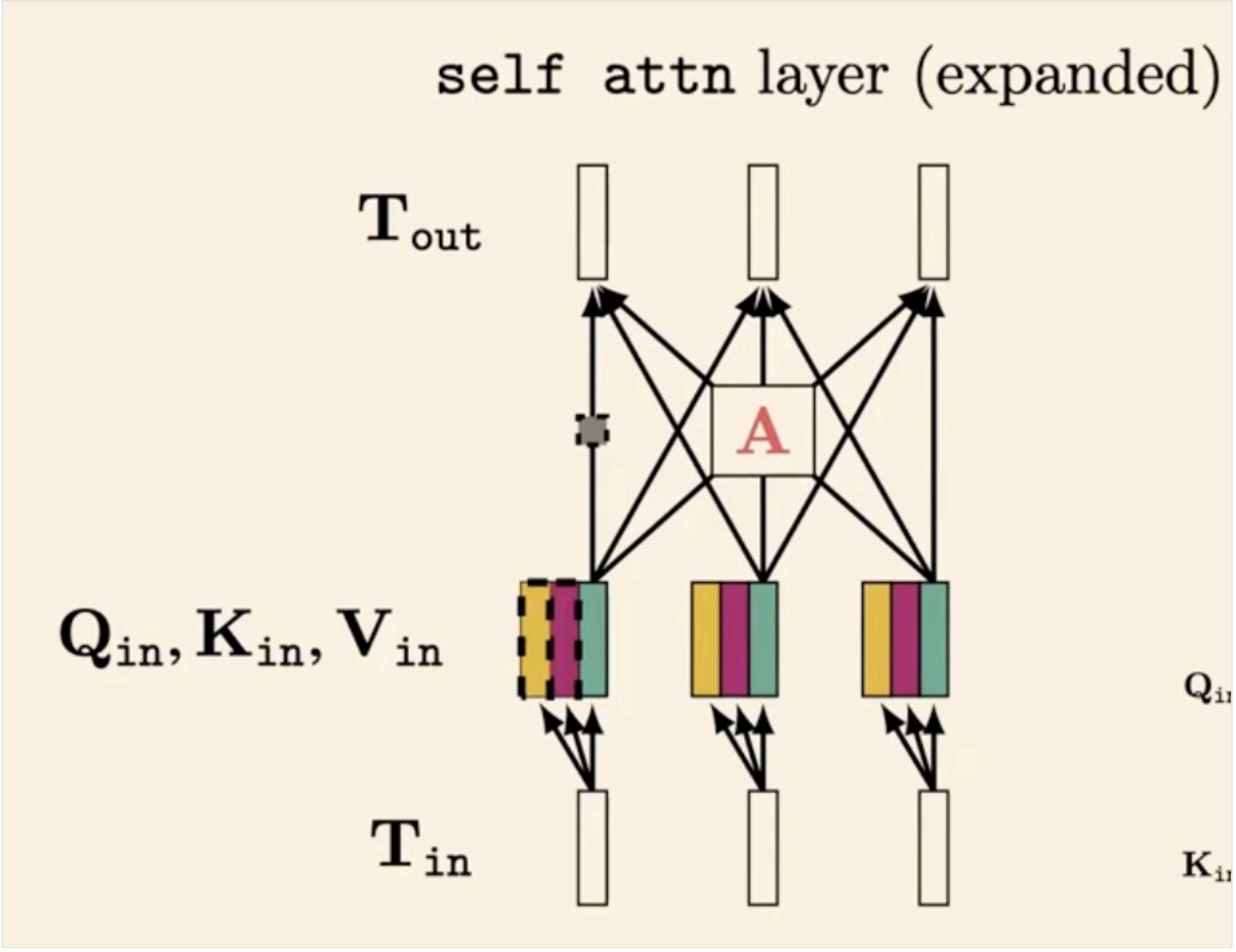


- 이 attention을 이용하면, 유사한 token끼리 segmentation할 수도 있음.


fc layer
T_{out}
W
T_{in}



Attention mathematics

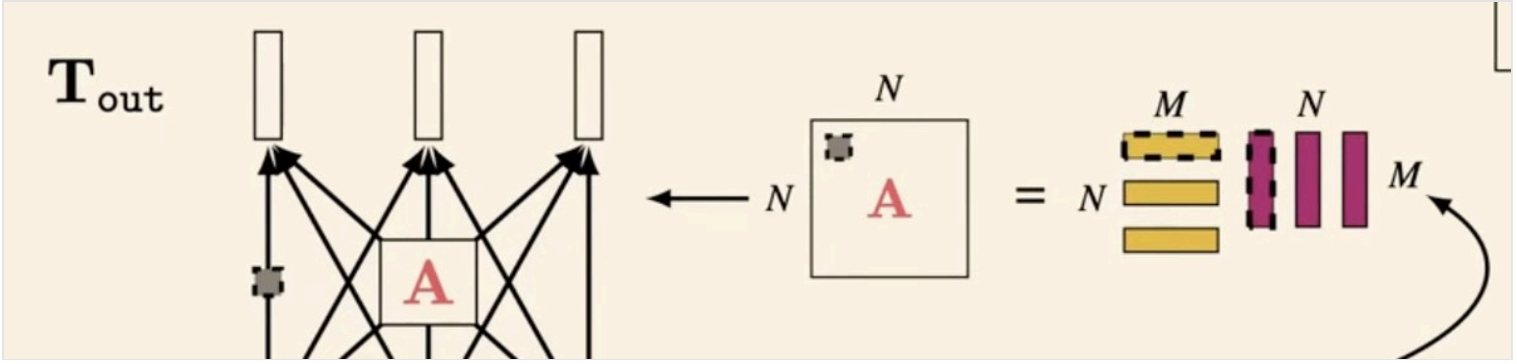


$$\mathbf{Q}_{\text{in}} = \begin{bmatrix} \mathbf{q}_1^T \\ \vdots \\ \mathbf{q}_N^T \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_q \mathbf{t}_1)^T \\ \vdots \\ (\mathbf{W}_q \mathbf{t}_N)^T \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_q^T \quad \triangleleft \quad \text{query matrix}$$

$$\mathbf{K}_{\text{in}} = \begin{bmatrix} \mathbf{k}_1^T \\ \vdots \\ \mathbf{k}_N^T \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_k \mathbf{t}_1)^T \\ \vdots \\ (\mathbf{W}_k \mathbf{t}_N)^T \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_k^T \quad \triangleleft \quad \text{key matrix}$$

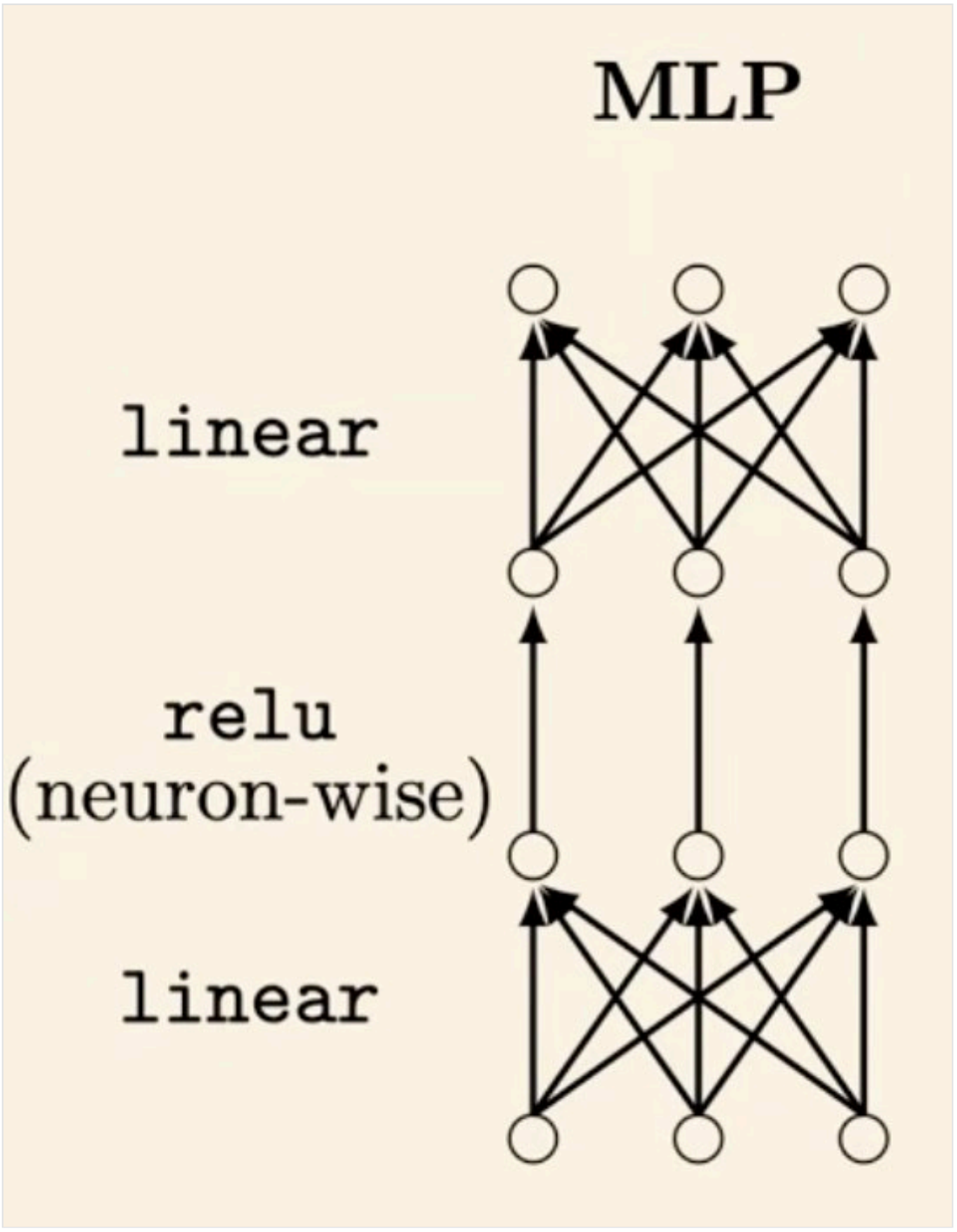
$$\mathbf{V}_{\text{in}} = \begin{bmatrix} \mathbf{v}_1^T \\ \vdots \\ \mathbf{v}_N^T \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_v \mathbf{t}_1)^T \\ \vdots \\ (\mathbf{W}_v \mathbf{t}_N)^T \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_v^T \quad \triangleleft \quad \text{value matrix}$$

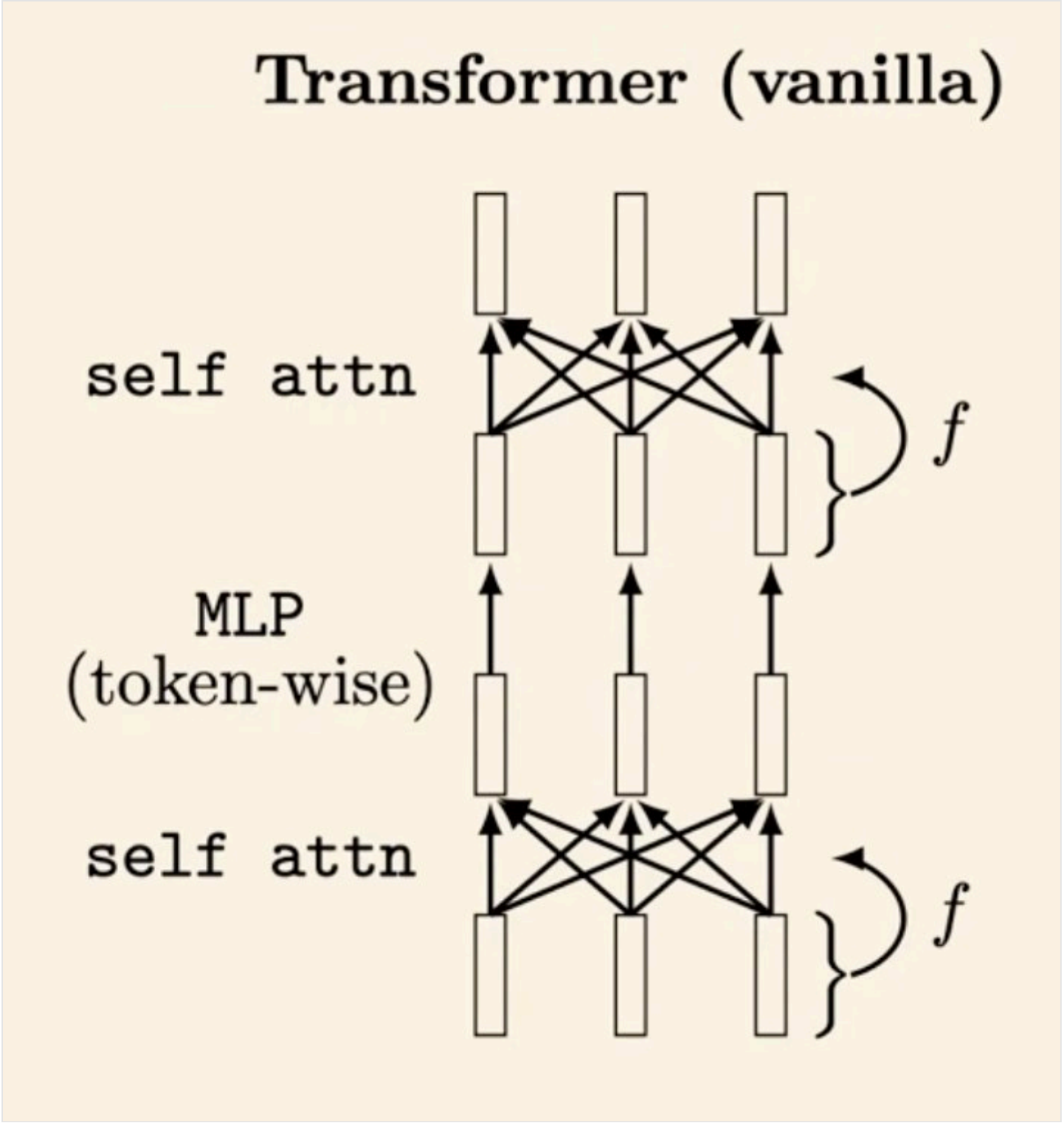
$$T_{out} = AV_{in}$$



$$A = f(T_{in}) = \mathrm{softmax}\left(\frac{Q_{in}K_{in}^{\top}}{\sqrt{m}}\right) \quad \triangleleft \quad \text{attention matrix}$$

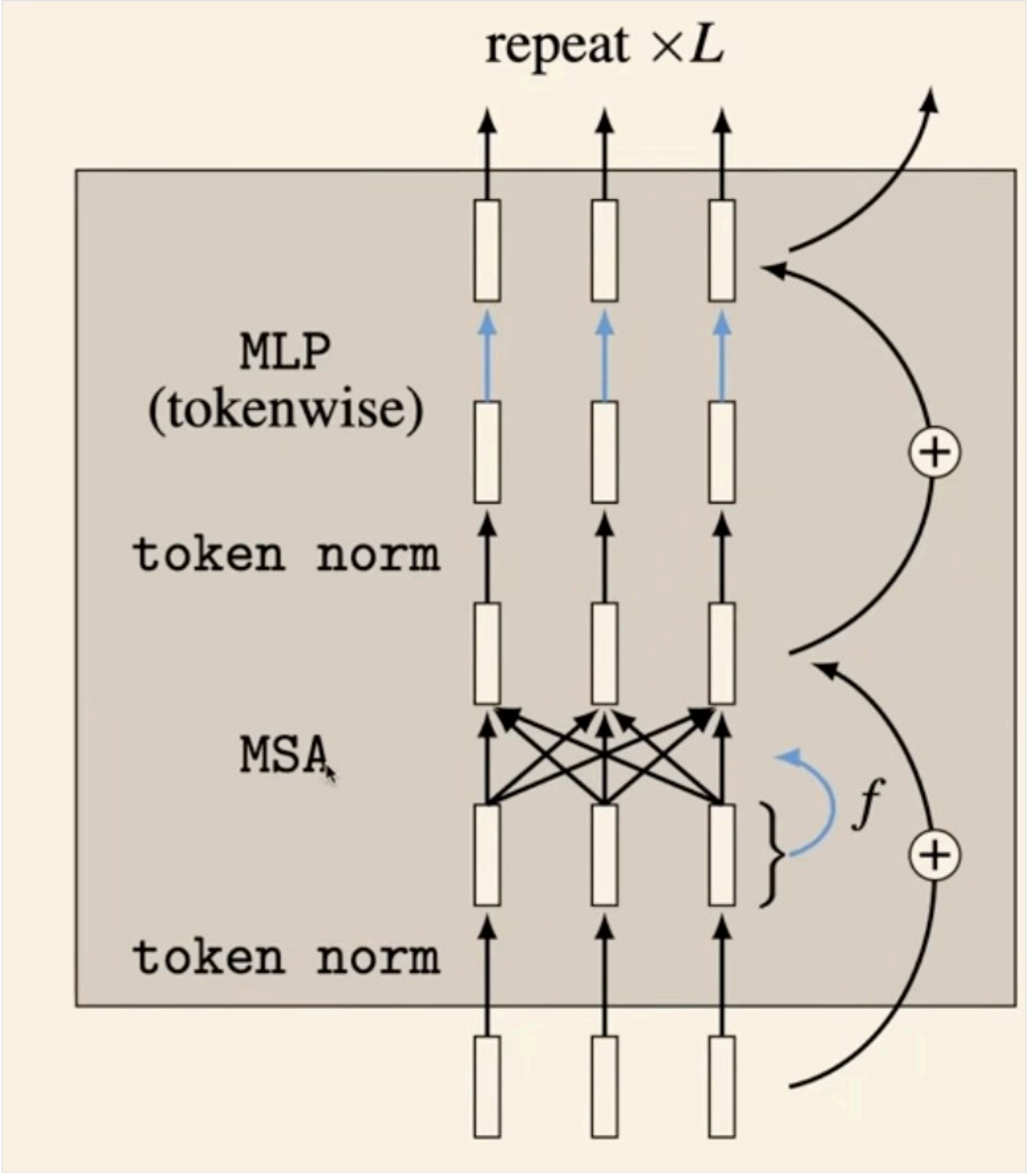
- Self-attention에서는 동일한 입력 시퀀스 T로부터 query, key, value를 각각 선형 투영(linear projection)하여 생성한다.
- $Q = T W_q$
- $K = T W_k$
- $V = T W_v$
- 여기서 W_q, W_k, W_v는 학습 가능한 projection 행렬이다.
- 입력 시퀀스의 토큰 수가 N이면, Q와 K를 곱하여 N×N 크기의 attention score matrix를 계산한다.
- $s = QK^T / \sqrt{d_k}$
- $A = \mathrm{softmax}(QK^T / \sqrt{d_k})$
- A의 크기는 N×N이며, 각 행은 하나의 query 토큰이 전체 key 토큰에 부여하는 attention weight를 나타낸다. Softmax는 각 query 행에서 key 방향으로 적용된다.
- 계산된 A를 V에 곱하면 self-attention의 출력이 된다.
- output = AV
- 즉, self-attention은 동일한 입력으로부터 Q, K, V를 생성하고, 각 토큰 사이의 관계를 나타내는 N×N attention matrix를 계산한 다음 이를 value에 적용하는 방식이다.


MLP
linear
relu
(neuron-wise)
linear



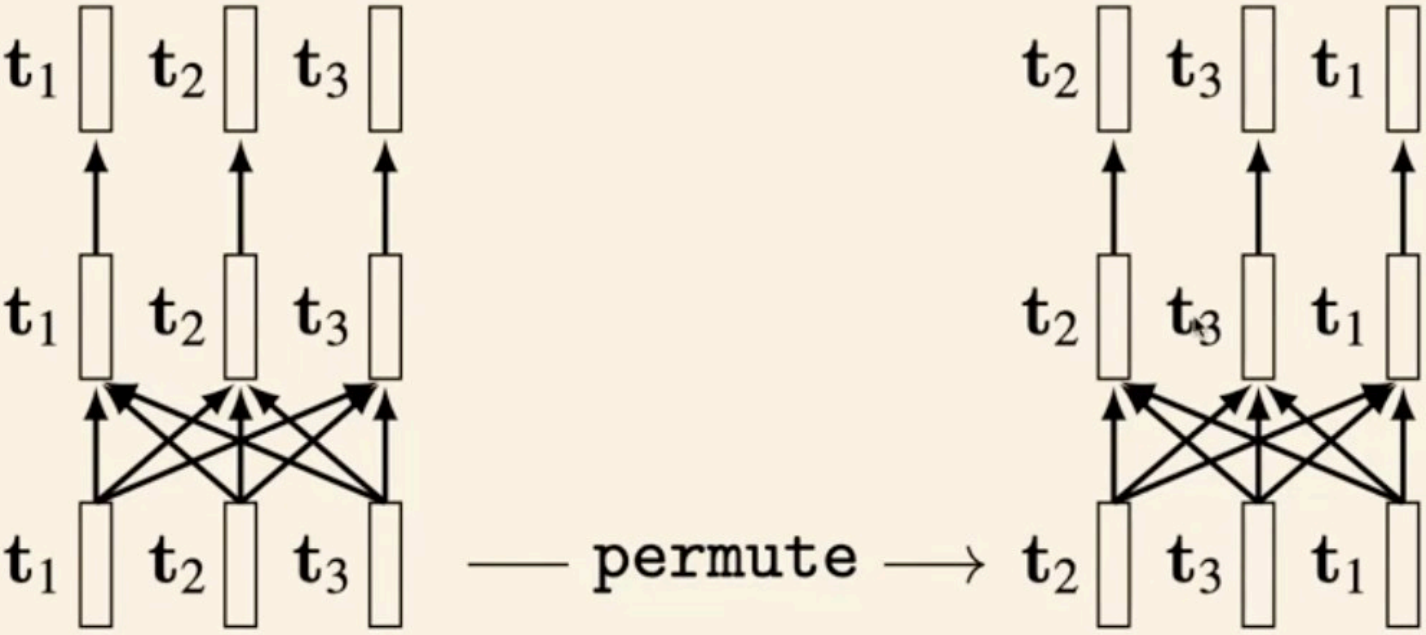
---

vision transformer(ViT)



---

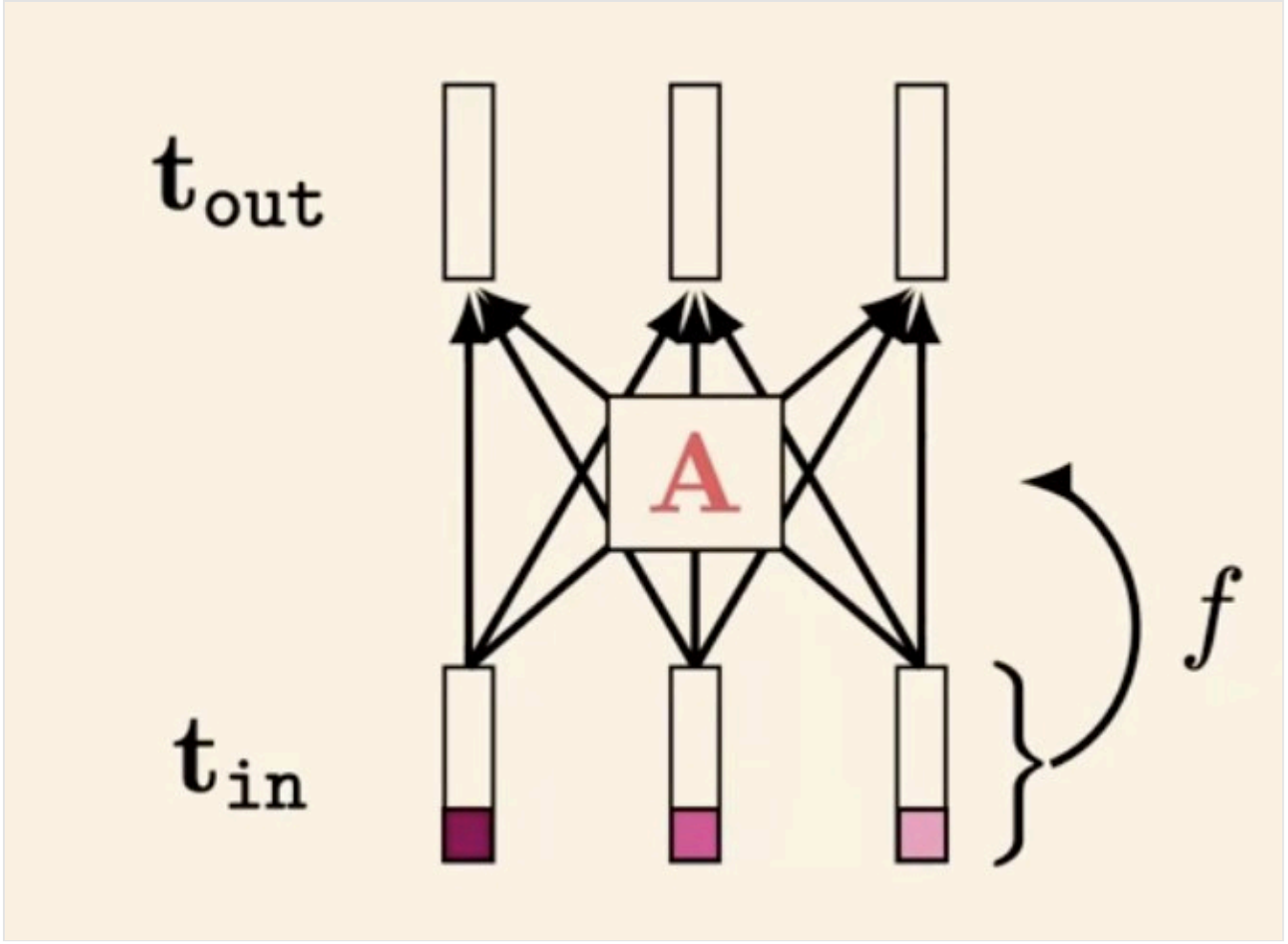
positional encoding



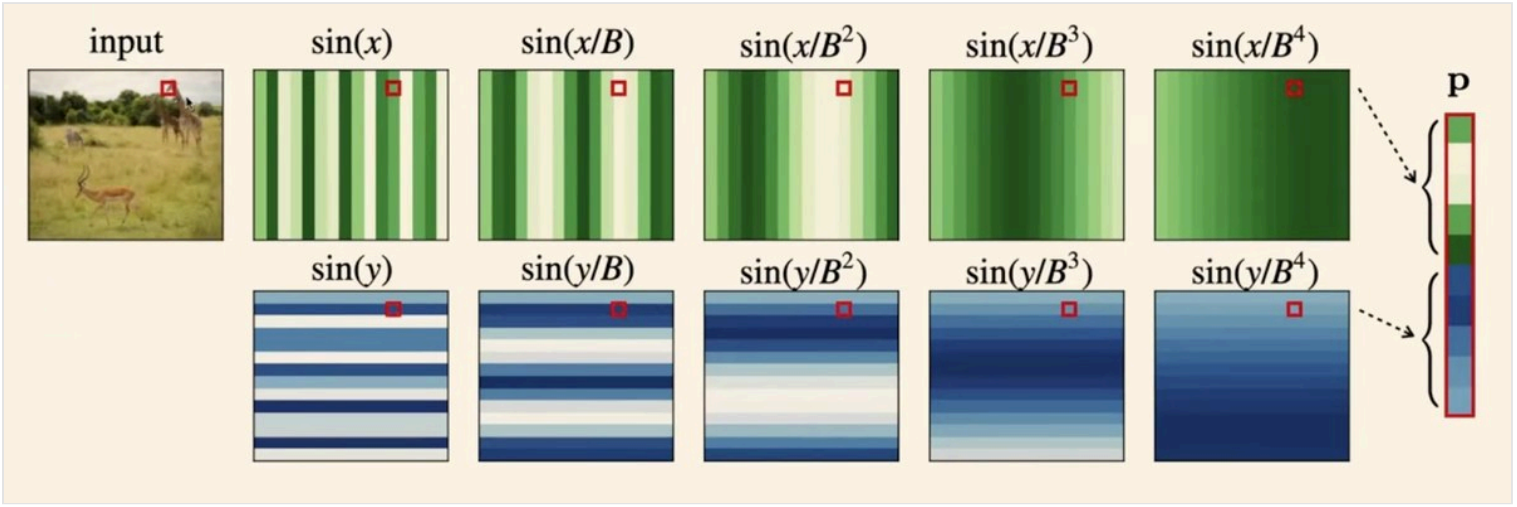
$$F_\theta(\texttt{permute}(\mathbf{T}_{\texttt{in}})) = \texttt{permute}(F_\theta(\mathbf{T}_{\texttt{in}}))$$

$$\texttt{attn}(\texttt{permute}(\mathbf{T}_{\texttt{in}})) = \texttt{permute}(\texttt{attn}(\mathbf{T}_{\texttt{in}}))$$

- token의 순서를 바꾸는 것이 아니라, 각 token vector에 위치 정보(positional encoding)를 더해 token들의 위치와 순서를 구분할 수 있게 함.
- position encoding은 token이 어느 곳에 있는지(이미지에서 위치/단어상 위치) 나타내는 것.



- token code vectors에다가, location information을 넣을 수 있음
  = **positional encoding**

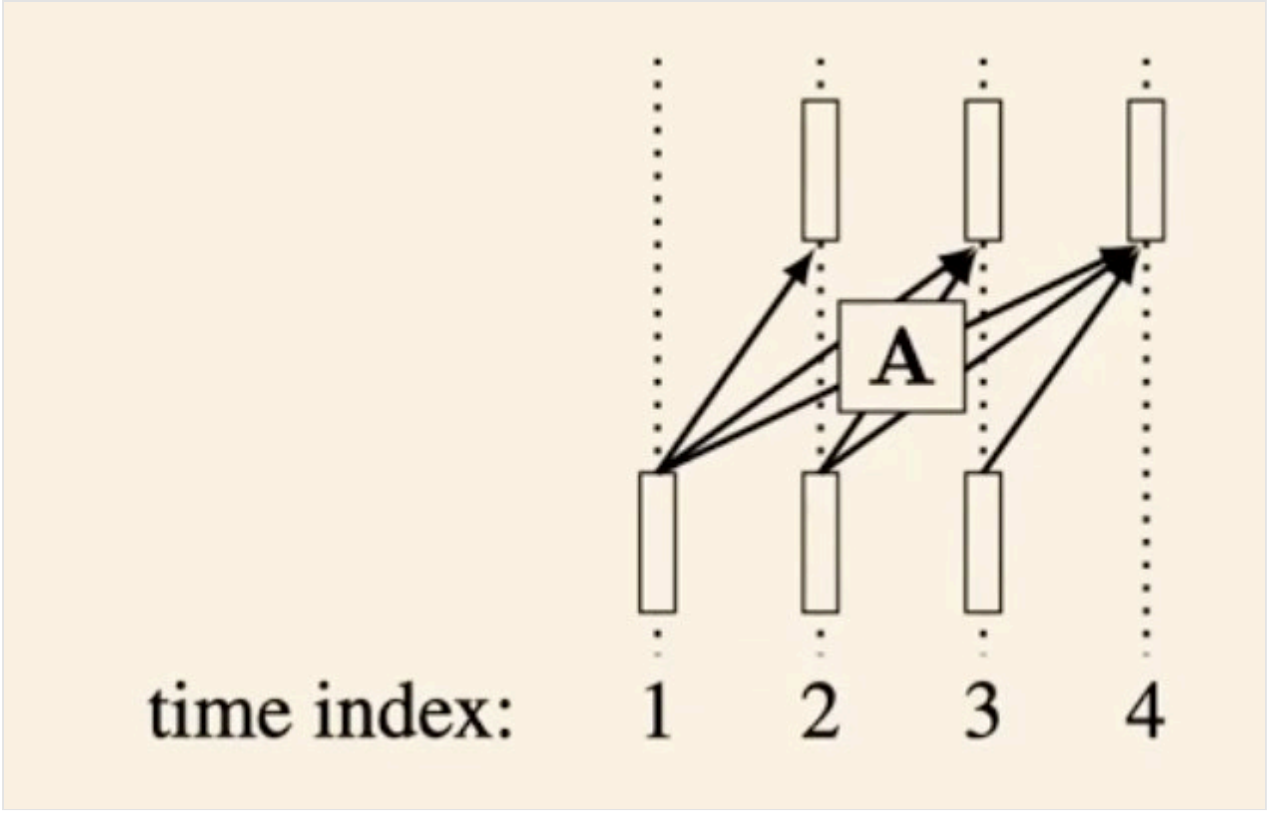


- 등등 여러 **positional encoding이 있음.**

Autoregressive models

LLM(Before)

- Once upon a time _____ → Predictor → _____
  Once ____ a time → Predictor → ____
  training set → Learner → Predictor possible word
- LLM(GPT)



- 이런 식으로 time index가 생김
- → prev token이 다음 time에 영향을 줌.

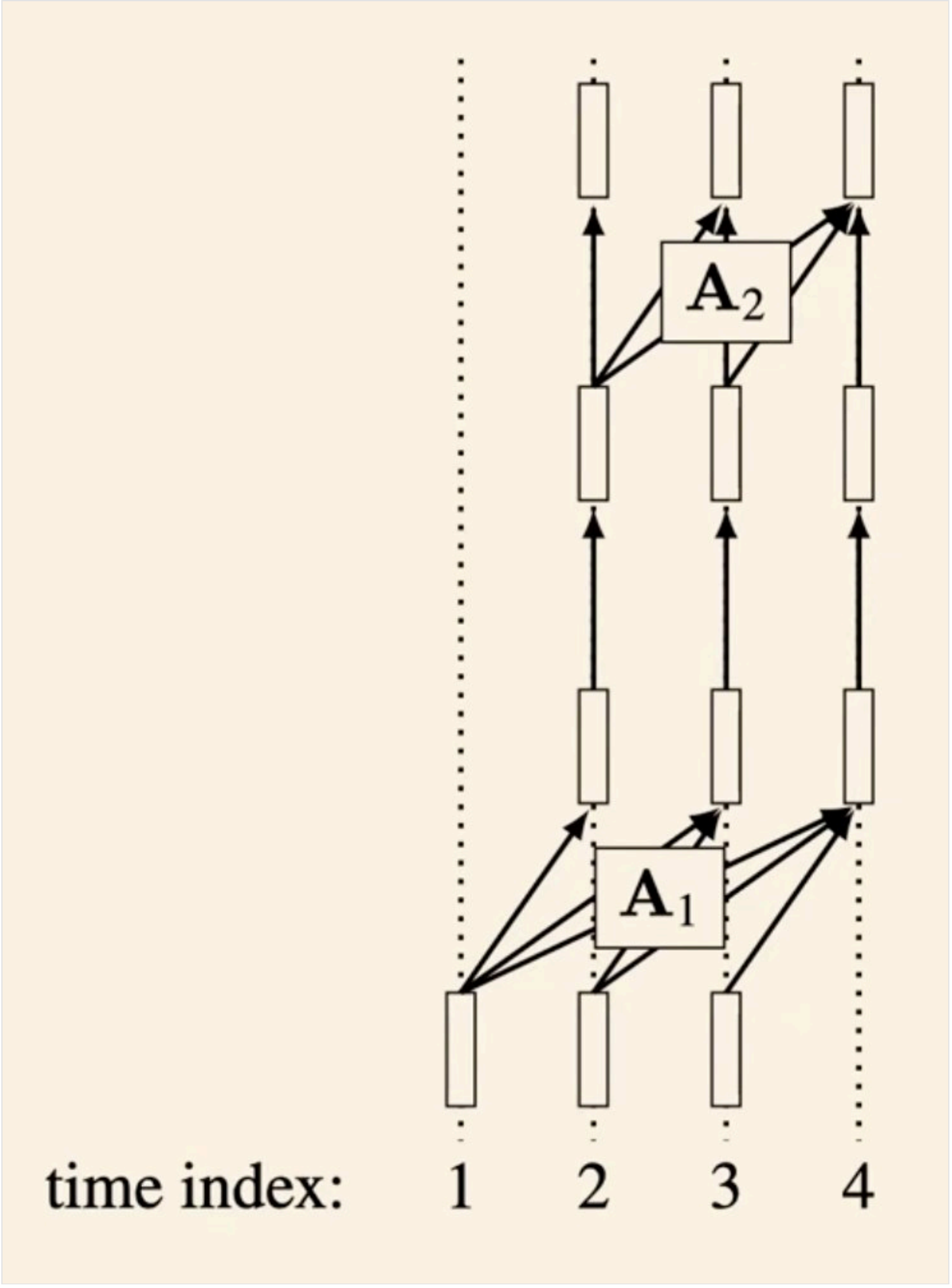
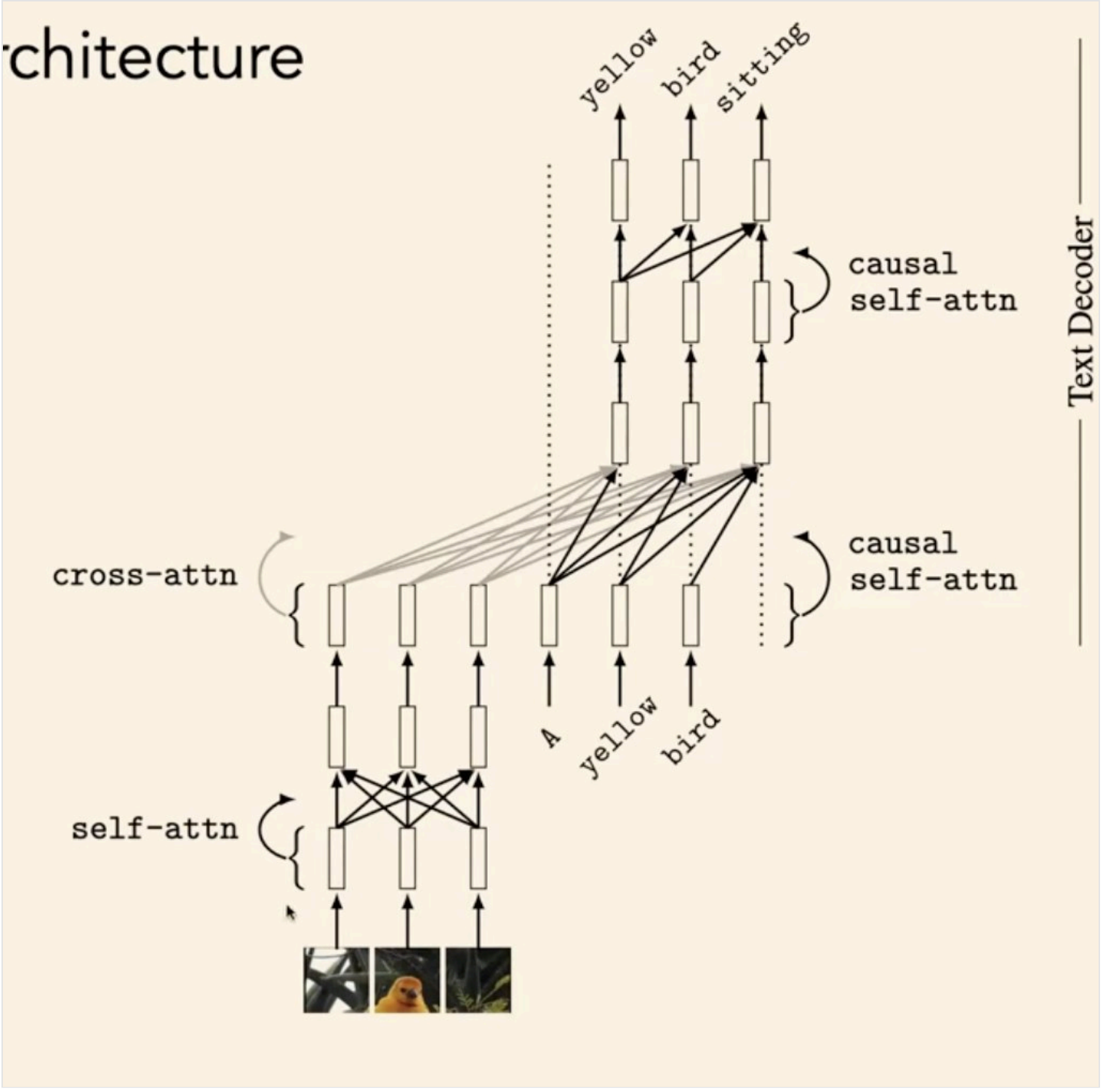
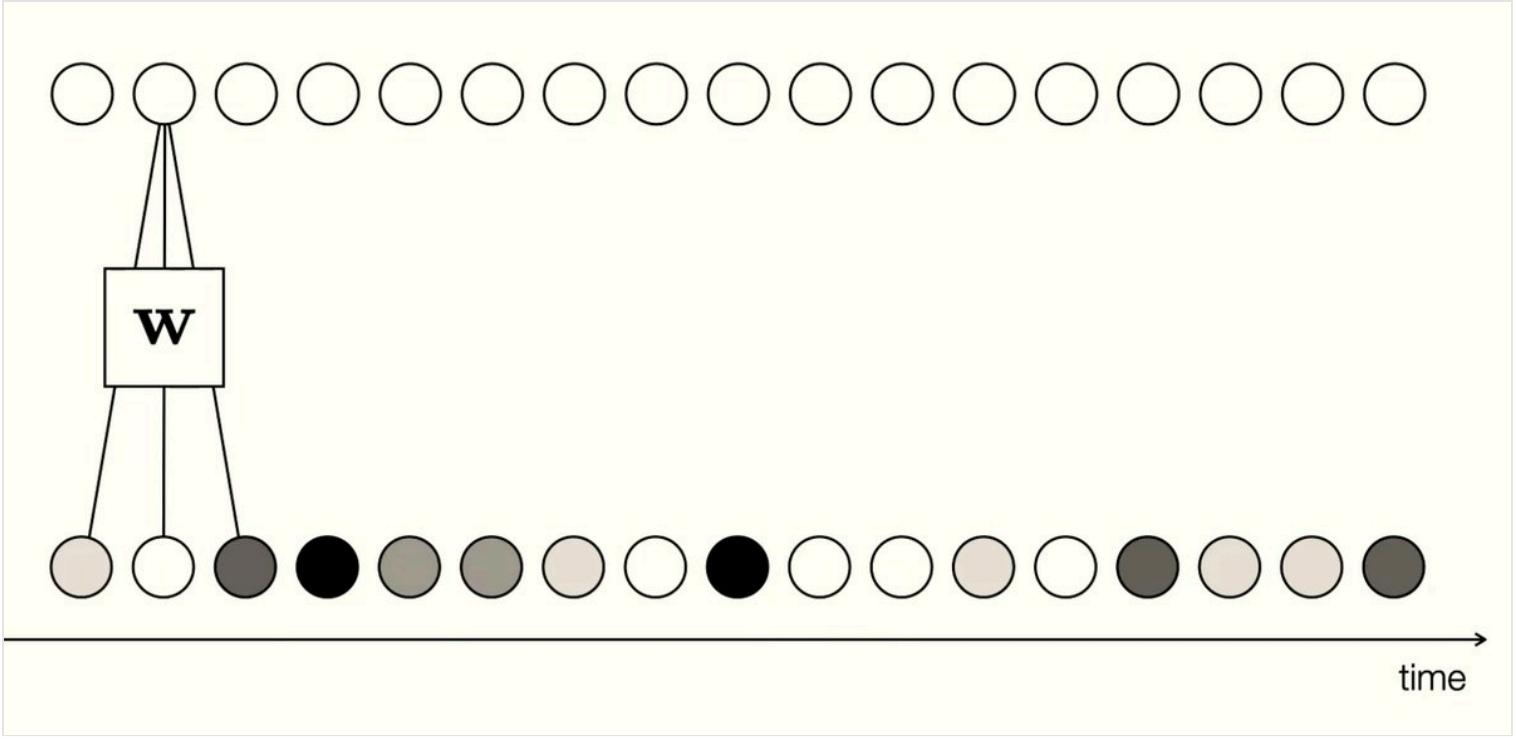


Image-to-text architecture

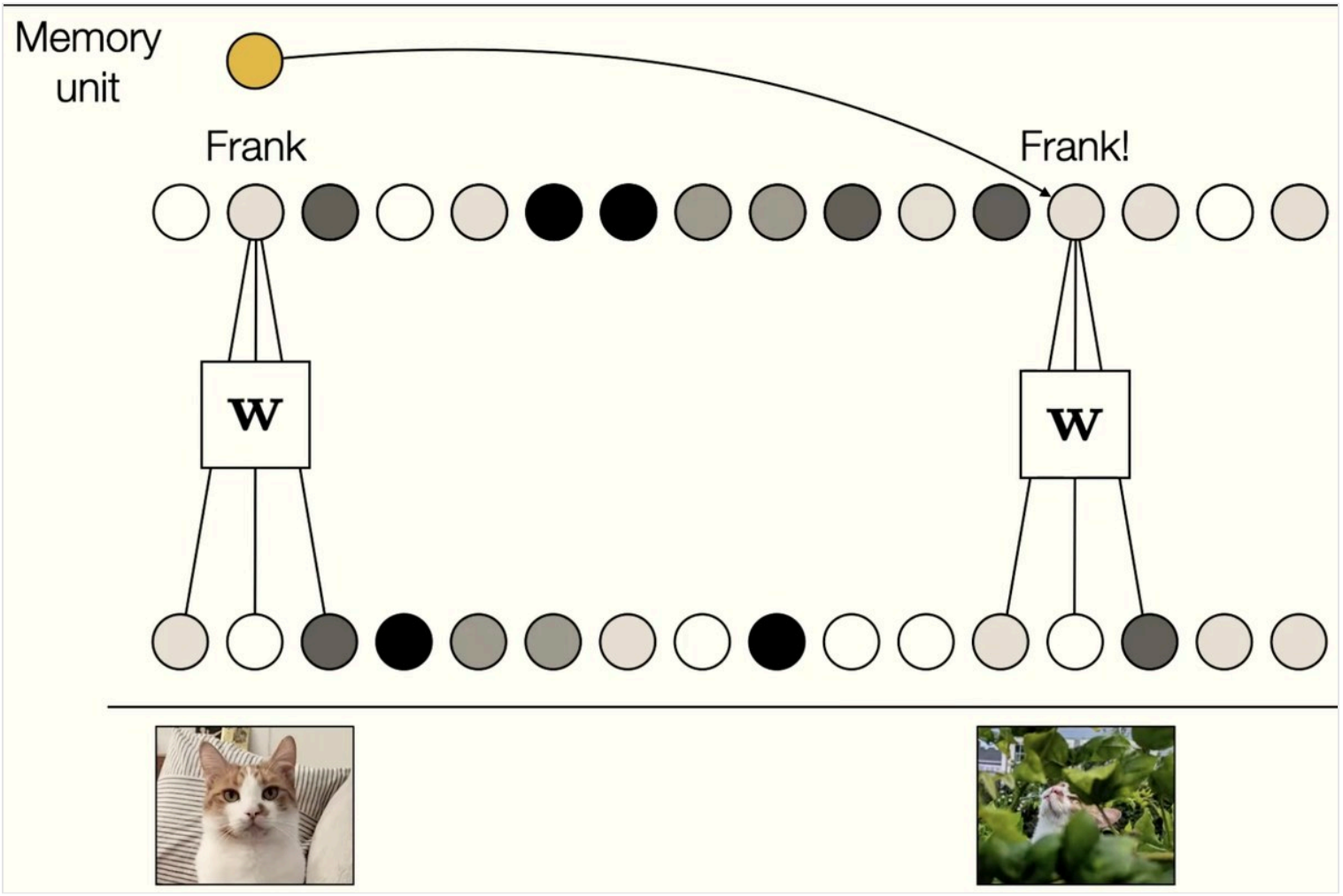


Lec. Memory

- 순차적인 것(time 등)을 모델링 하고 싶음.

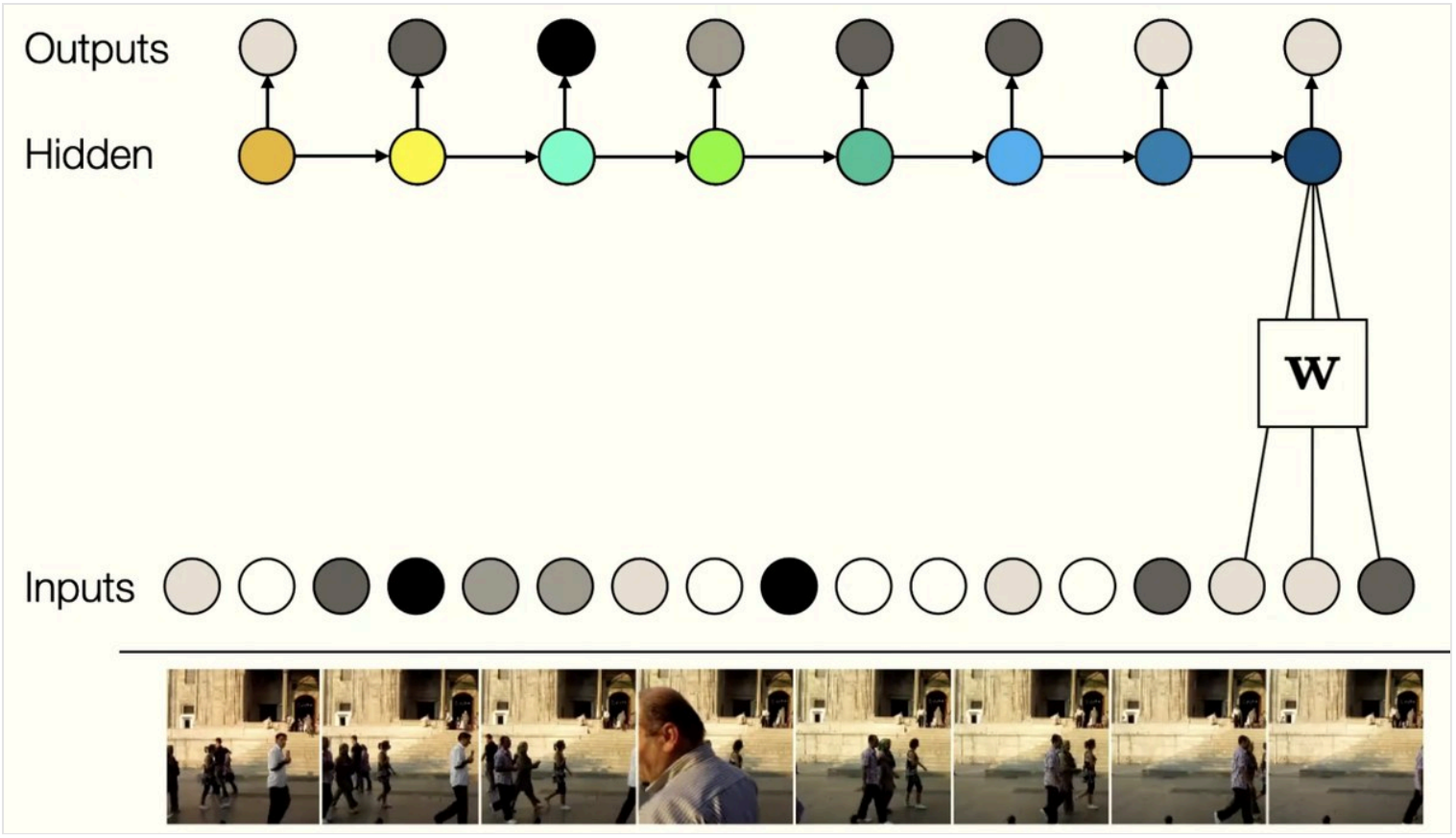


- 시간에 따라서, Convolutions할 수 있음.

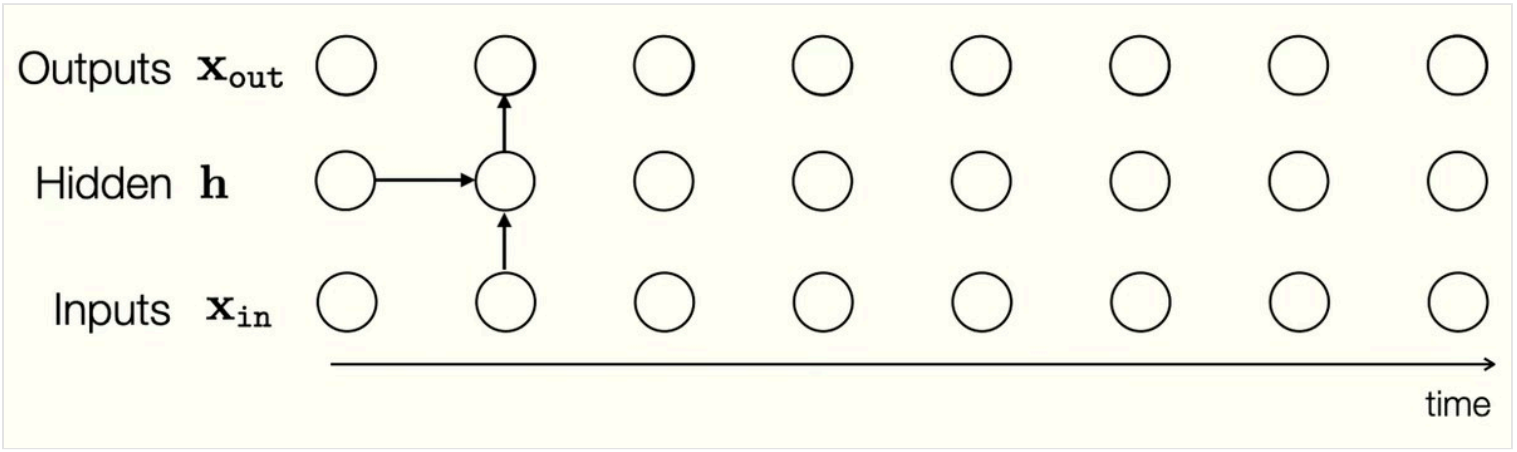


- 그런데 시간이 길어질수록 → 시간의 초반 부분을 기억하기 어려워짐.
  초반의 정보(예: 고양이)가 제대로 인식되었더라도,
  후반에 이 정보를 잊고, 다시 인식하는 과정에서 다른 것으로 인식할 수 있음(예: 고양이인 걸 잊고, 호랑이라고 해버림)

- Recurrent Neural Networks(RNNs)

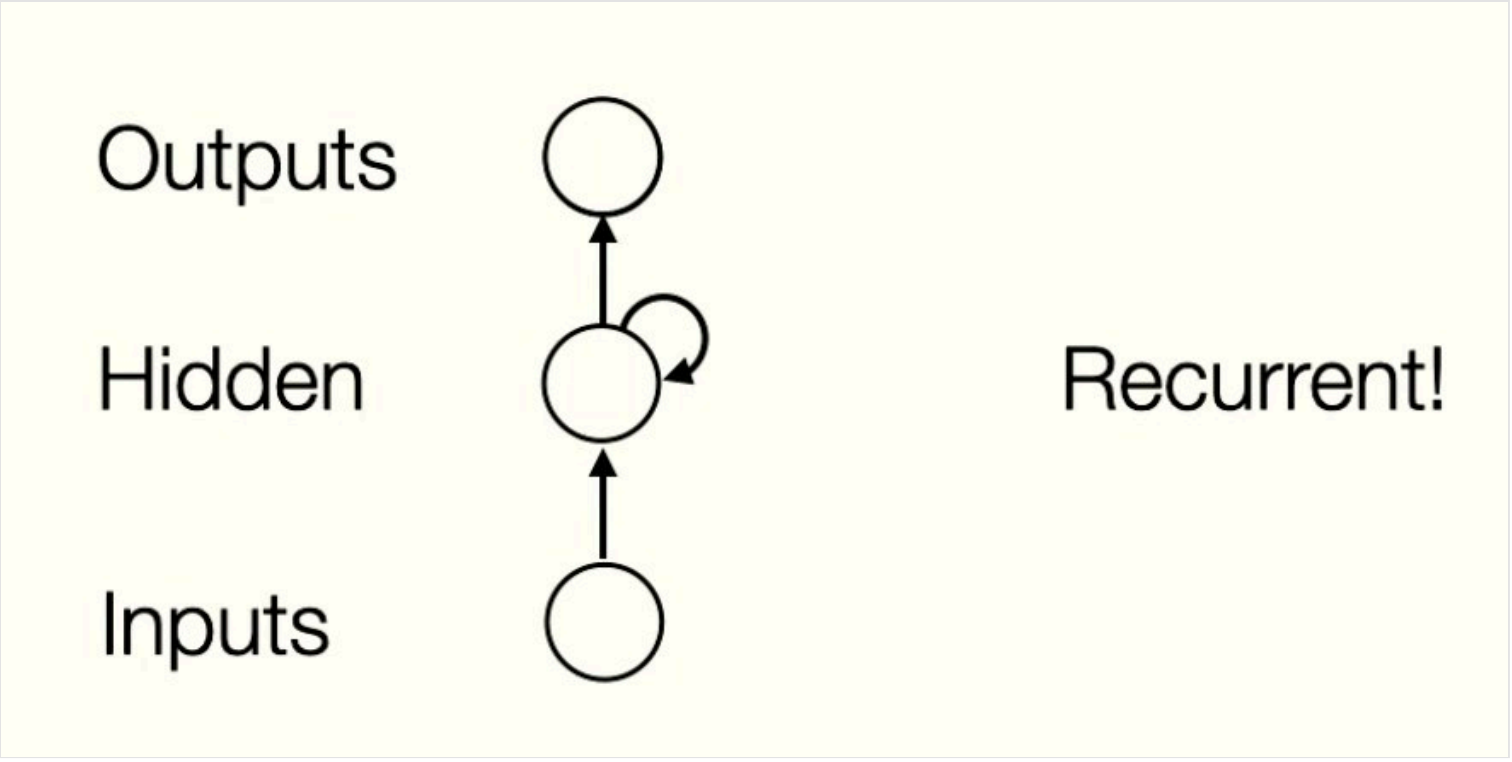


- hidden unit을 이용해서, memory까지 얻을 수 있음

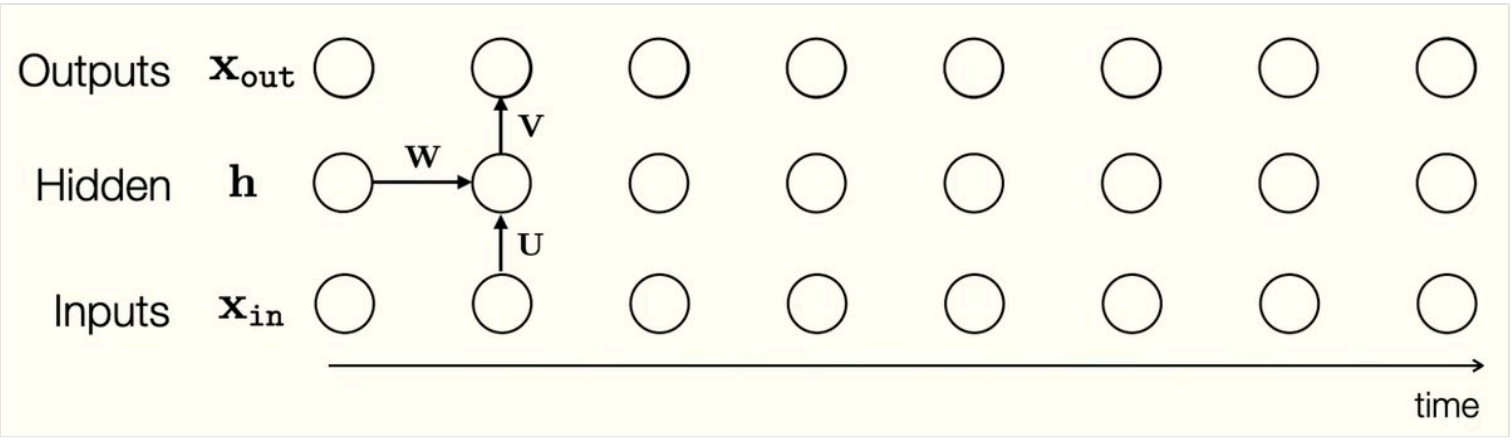


- $h_t = f(h_{t-1}, x_{in}[t])$로 표현할 수 있음. f is fix(independ on time)

$$x_{out}[t] = g(h_t)$$



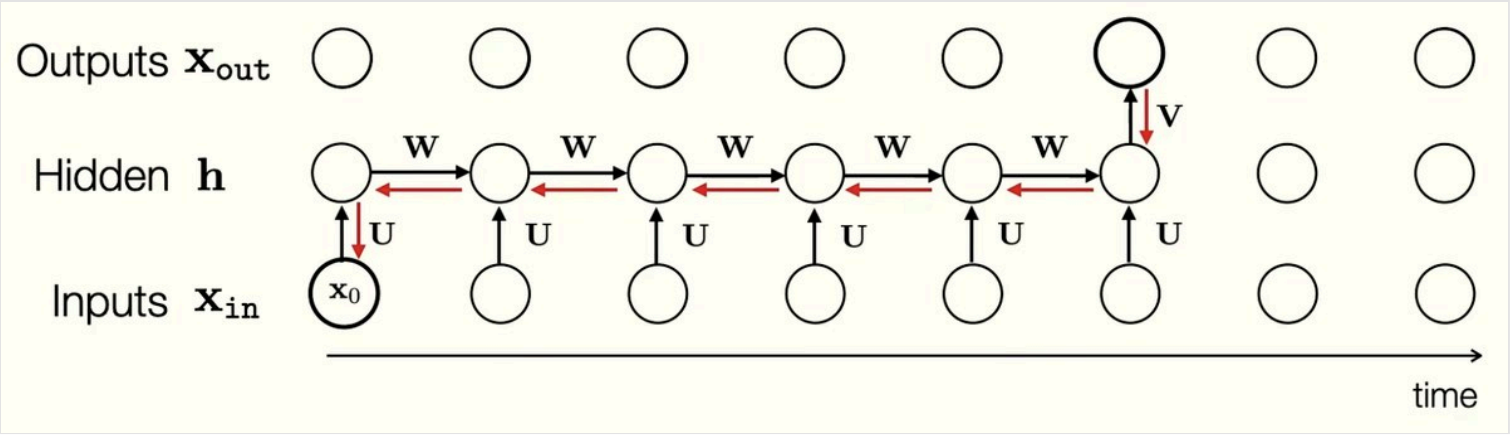
- hidden state에 loop가 있는 형태라고 볼 수 있음



- RNN의 hidden state 갱신은 이전 hidden state와 현재 입력에 대한 선형 변환에 비선형 활성화 함수(tanh 등)를 결합한 형태로 표현할 수 있음.
- 즉, linear layer만으로 모두 표현되는 것이 아니라, 선형 변환 결과에 비선형 활성화를 적용하여 새로운 hidden state를 계산함.

$$h_t = \sigma_1(Wh_{t-1} + Ux_{in}[t] + b)$$
$$x_{out}[t] = \sigma_2(Vh_t + c)$$

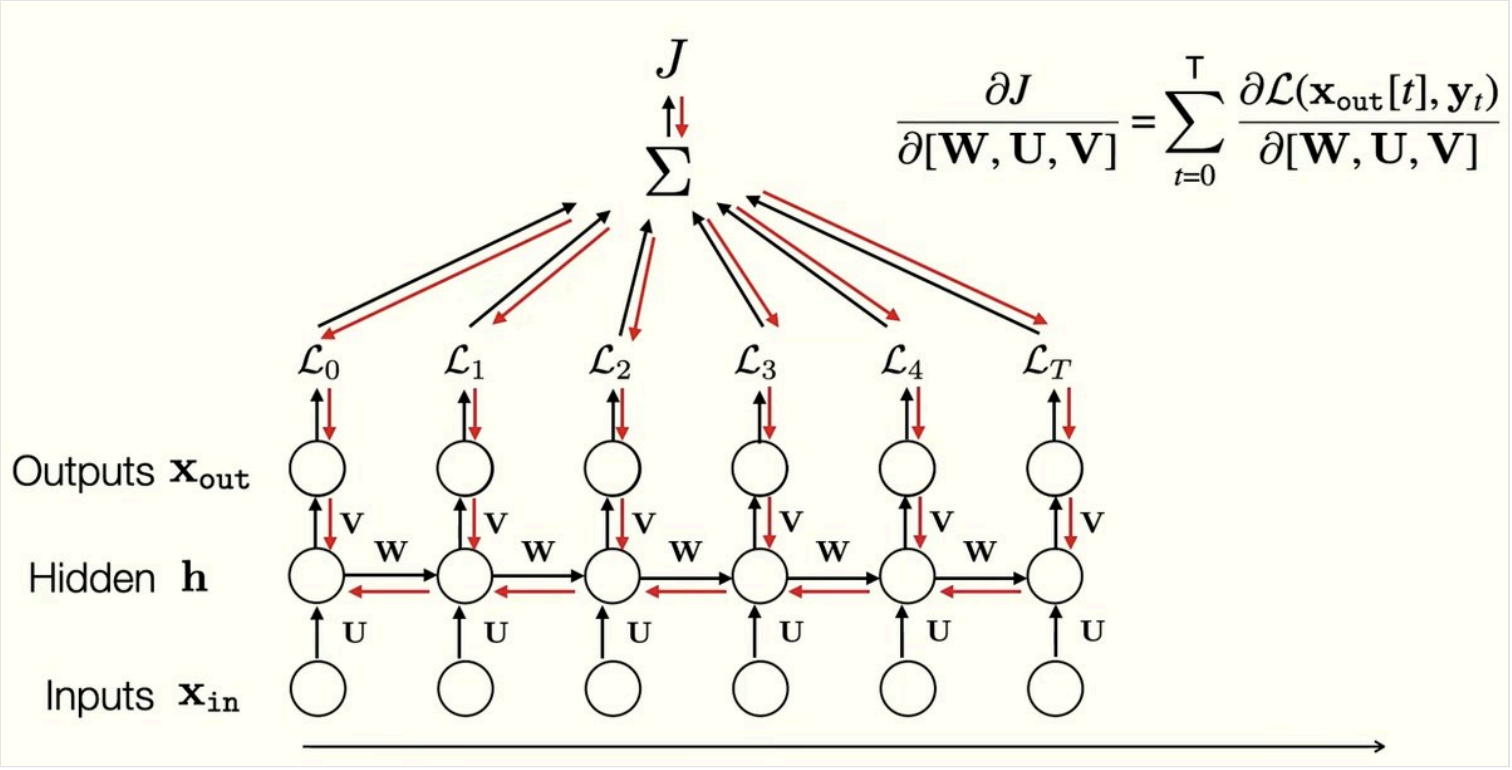
Backprop through time



- 시간에 따라 가는 그래프를 거꾸로 따라가면 됨.

$$\frac{\partial \mathbf{x}_{\text{out}}[t]}{\partial \mathbf{x}_{\text{in}}[0]} = \frac{\partial \mathbf{x}_{\text{out}}[t]}{\partial \mathbf{h}_T} \frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_{T-1}} \cdots \frac{\partial \mathbf{h}_1}{\partial \mathbf{h}_0} \frac{\partial \mathbf{h}_0}{\partial \mathbf{x}_{\text{in}}[0]}$$

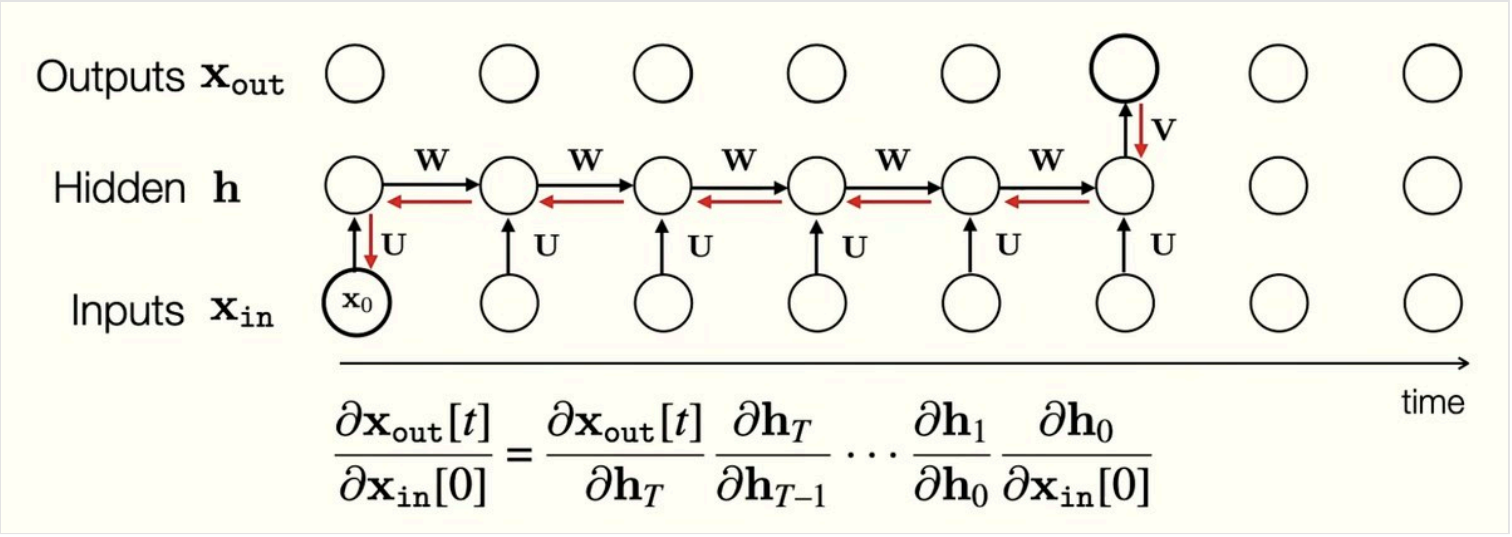
- 여기서, W가 고정되어 있음.



-

---

why not remember everything?

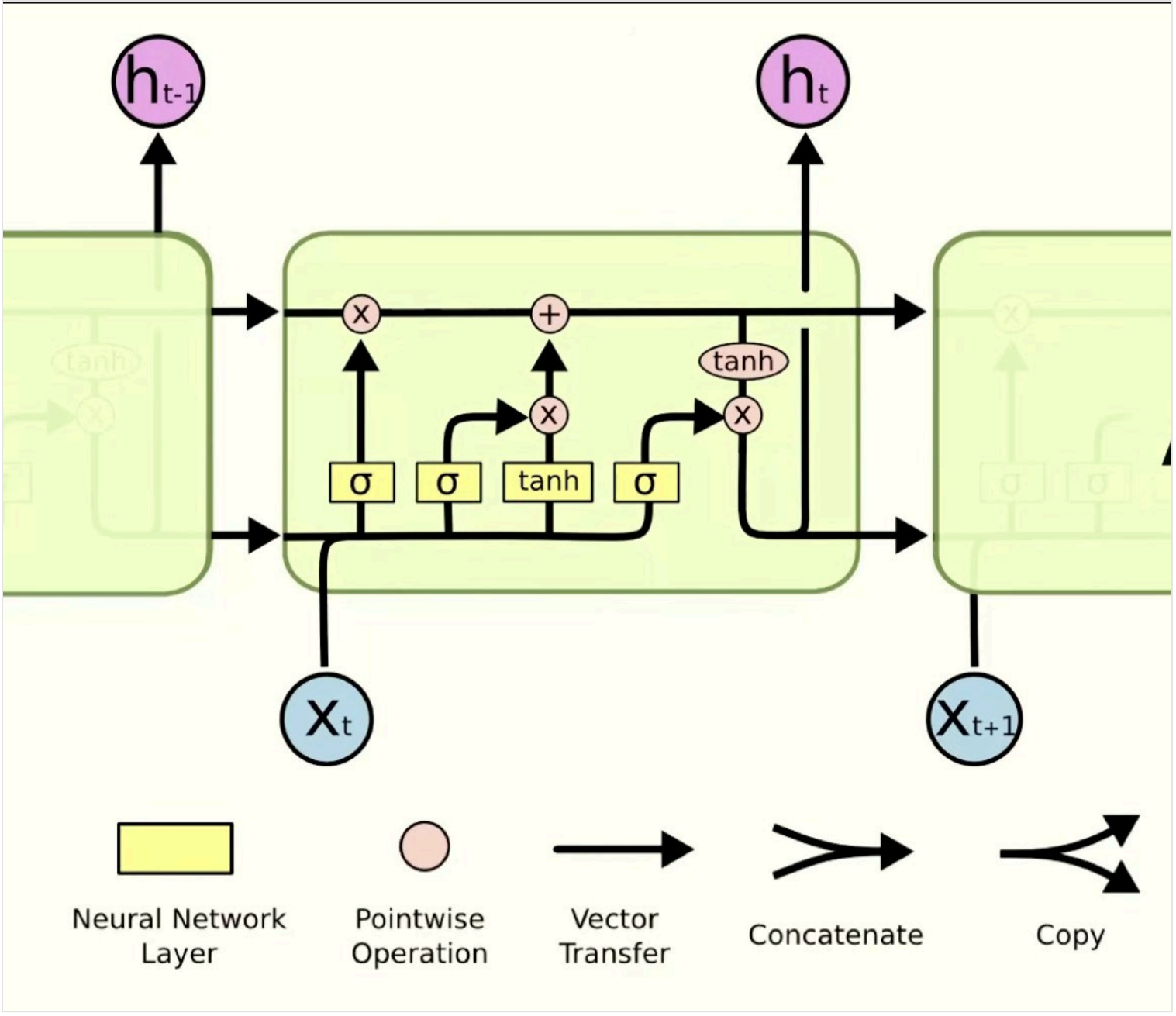


- time 차이가 큰 곳의 결과에 영향을 주려면, 많은 W를 거쳐야 함.

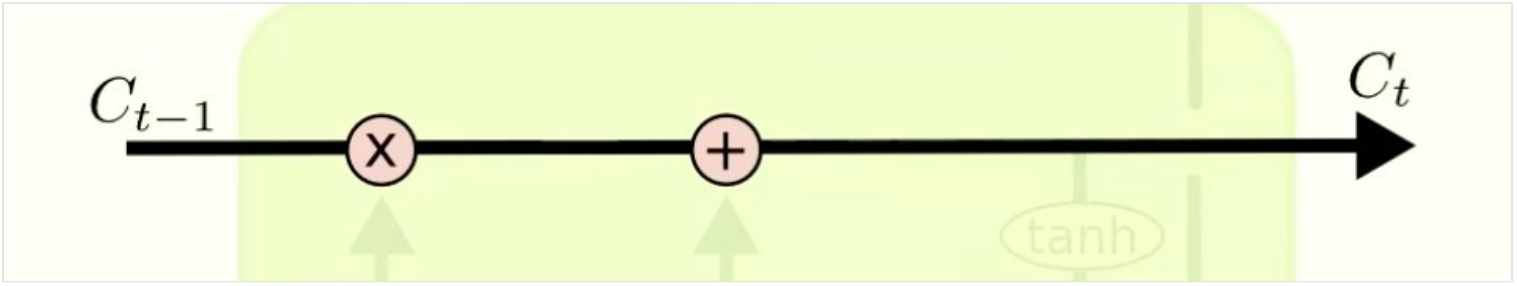
$$h_1 = Wh_0 + u_{x_1}$$
$$h_2 = W(Wh_0 + u_{x_1}) + u_{x_2}$$

- W^t 항을 가지게 됨.
- RNN에서는 시간 단계를 거치며 recurrent weight가 반복적으로 적용되므로, 역전파 과정에서 기울기 소실(vanishing gradient) 또는 기울기 폭발(exploding gradient)이 발생할 수 있다.
- 스칼라 W를 반복해서 곱한다고 가정하면, |W| < 1일 때 값이 0으로 수렴하고 |W| > 1일 때 값이 커질 수 있다. 그러나 이는 현상을 설명하기 위한 단순한 스칼라 직관이며, 실제 RNN의 행렬 연산에 그대로 적용할 수는 없다.
- 행렬 W의 반복적인 효과는 스펙트럼 반경(spectral radius)과 노름(norm) 등을 고려하여 분석해야 한다. 특히 실제 BPTT(Backpropagation Through Time)의 기울기 전파는 recurrent weight만이 아니라 활성화 함수의 도함수까지 포함한 야코비안(Jacobian)의 곱에 의해 결정된다.
- $J_t = \text{diag}(\phi'(a_t))W$
- 여기서 a_t는 시간 t의 활성화 함수 입력이고, ϕ'는 활성화 함수의 도함수이다. 여러 시간 단계에 걸친 기울기는 J_t, J_{t-1}, … 등의 연속적인 곱에 영향을 받으며, 그 크기가 지나치게 작아지거나 커질 수 있다.
- 따라서 단순히 W를 1에 가깝게 만드는 것만으로 기울기 문제를 해결할 수는 없다. 이러한 장기 의존성 학습의 어려움을 완화하기 위해 cell state와 gating mechanism을 사용하는 LSTM이 도입되었다.
- ⇒ **LSTM을 사용하기 시작함.**

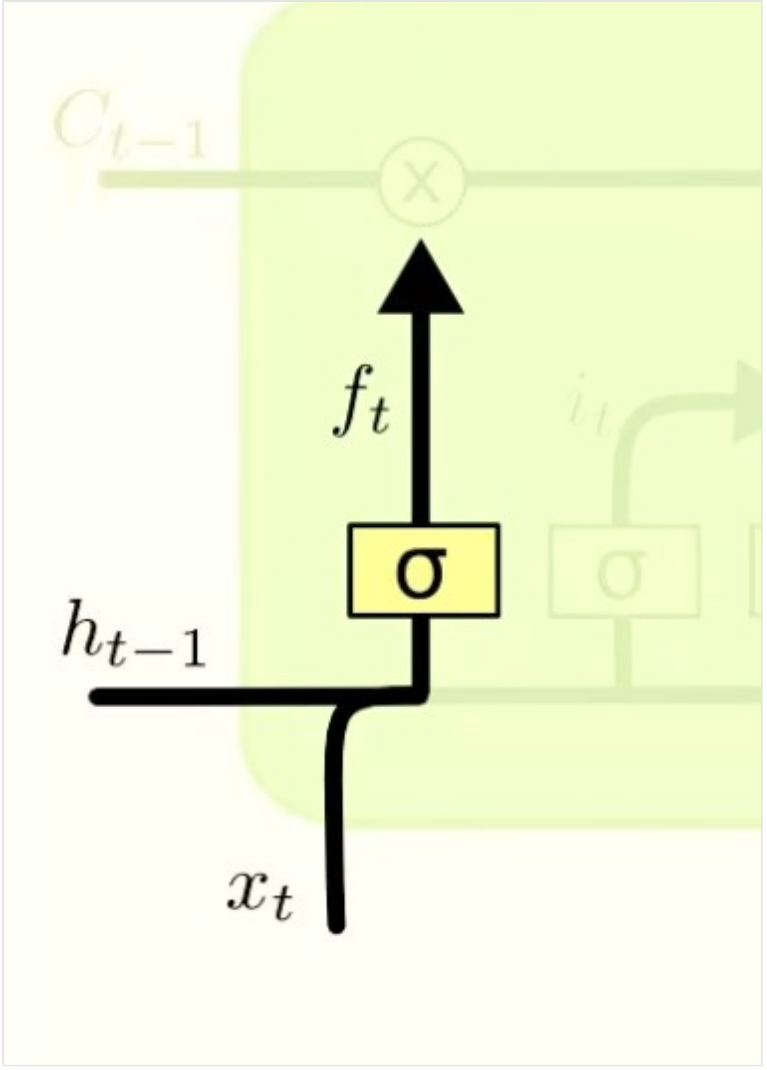
LSTM



- 저장할 것 / 잊을 것을 구분하는 장치가 있음.

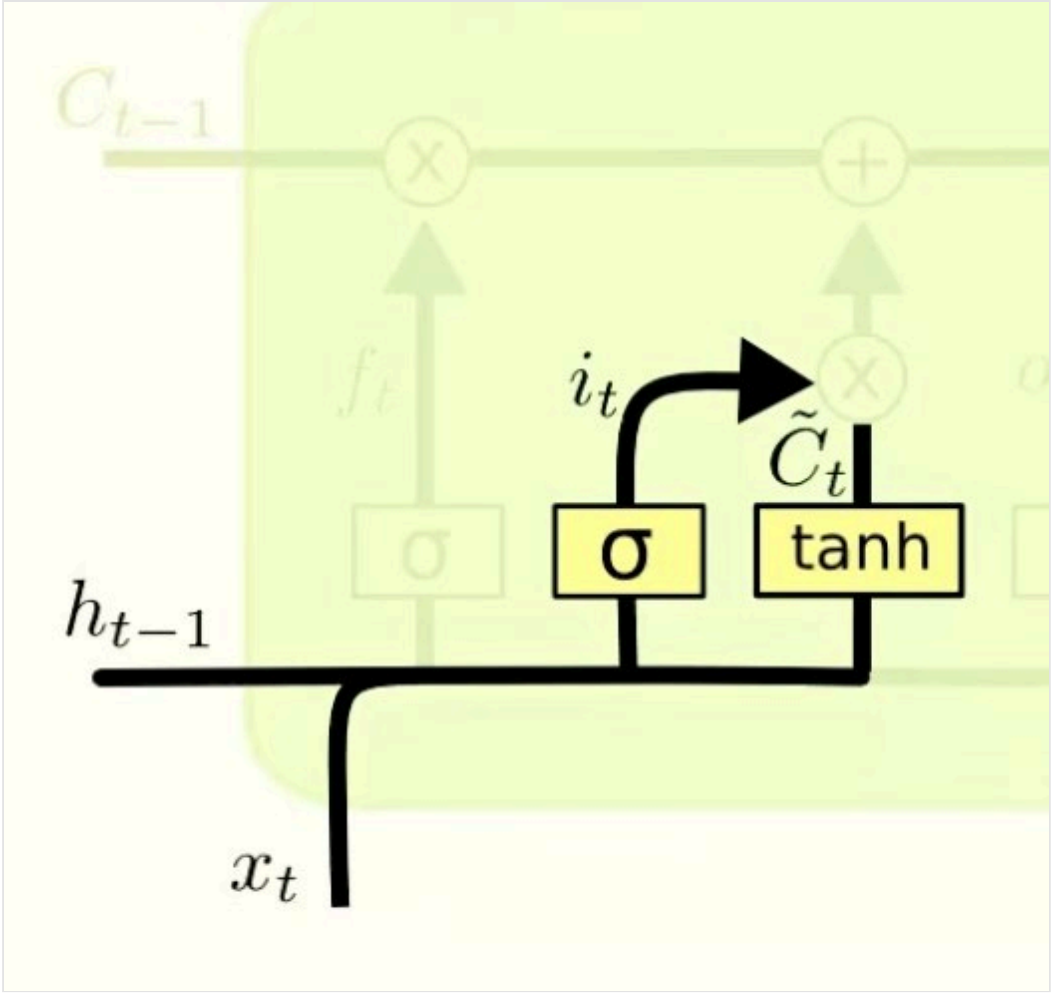


- C_t = Cell state → Long-term Memory 역할
  → 무엇을 잊고, 무엇을 추가할지 결정하는 부분에 의해 조절됨.

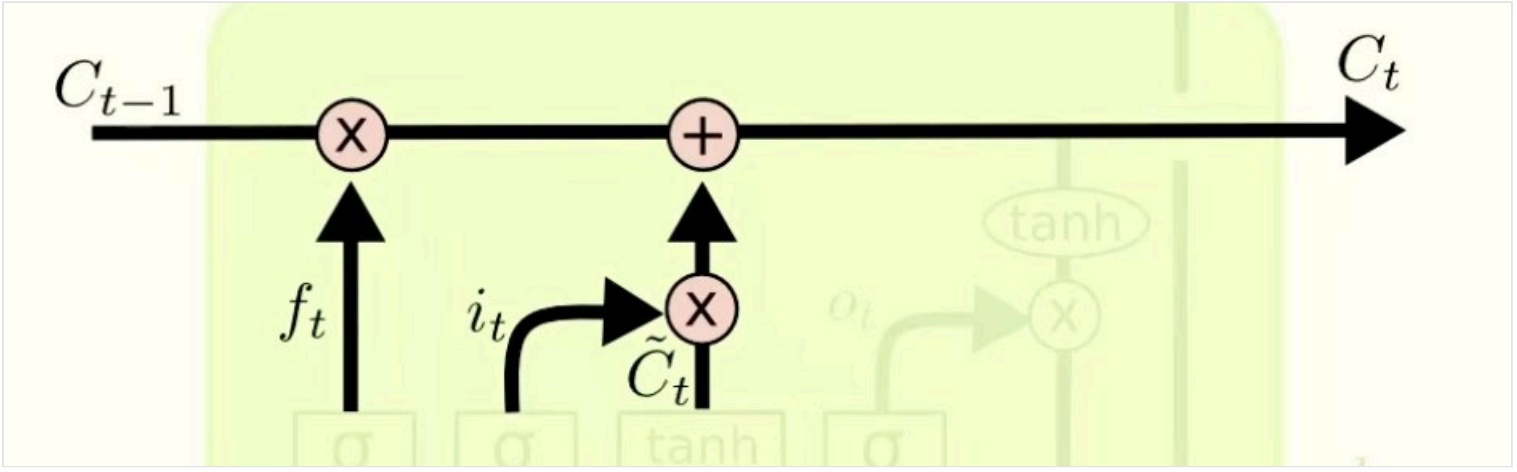


$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

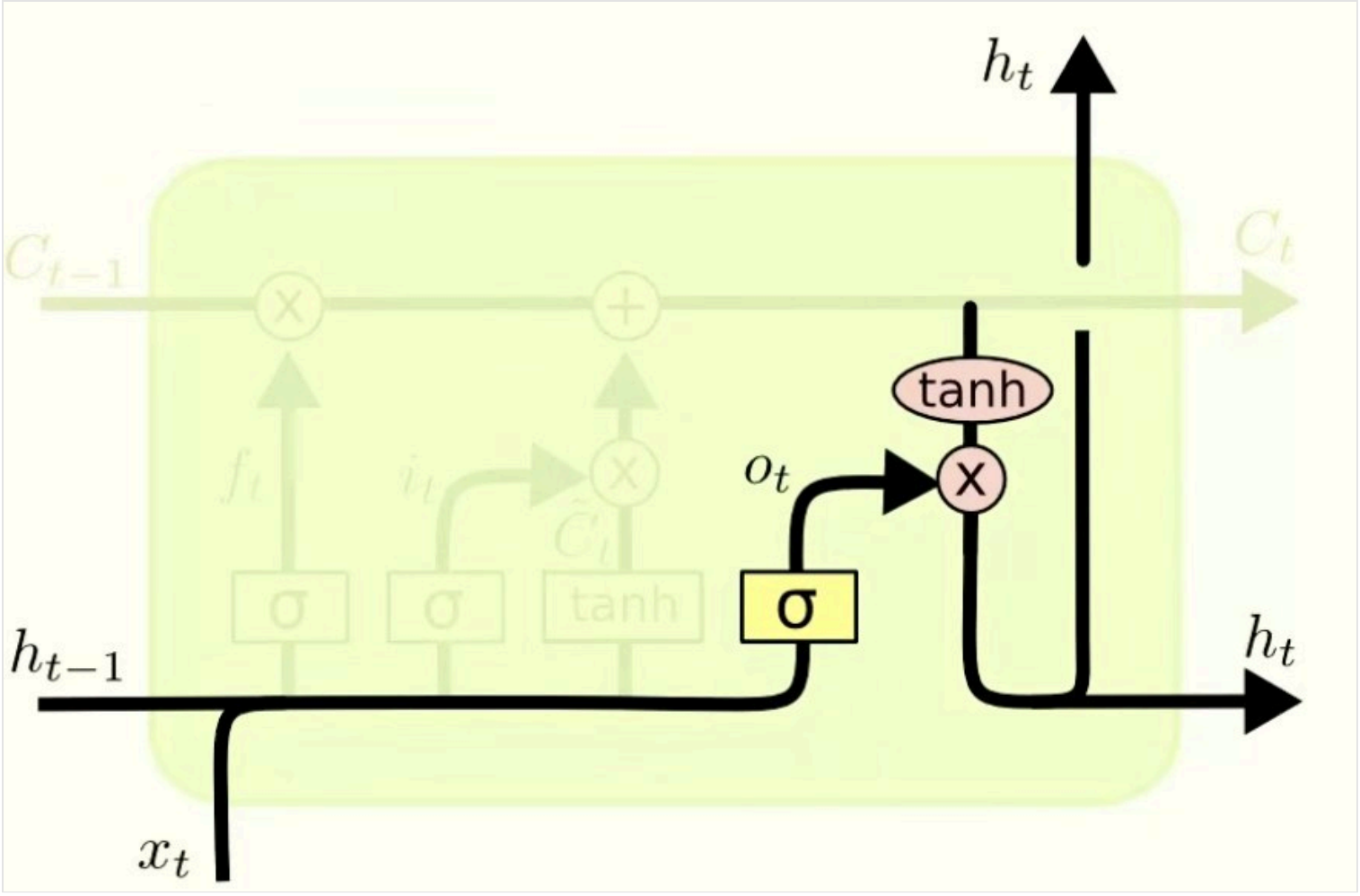
- 기억하고 싶으면 → 높은 값.
  잊고 싶으면 → 낮은 값
  → on/off 스위치 역할을 함.
  ⇒ 기억하고/잊을 것을 결정하는 역할



- $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$
  → which indices to write to
- $\tilde{C}_t = tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$
  → what to write to those indices
- cell state에 추가할 새로운 정보를 결정



- 실제로 잊는 과정을 거침.
- $C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$
  → 이전의 정보를 지우거나 유지하고 / 새로운 정보를 추가하는 과정

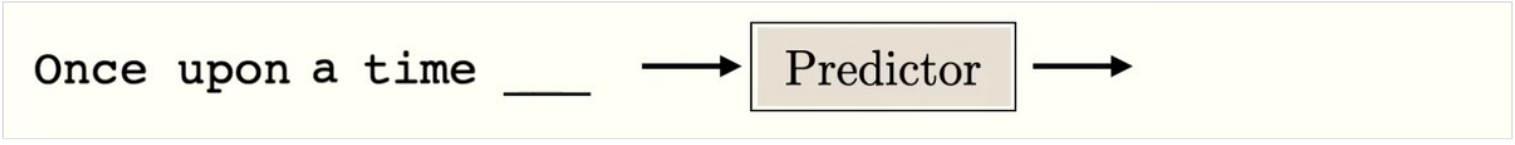


$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$
$$h_t = o_t * tanh(C_t)$$

- 여기서는 어떤 것을 hidden state로 출력할지를 정하게 됨.
- transformer처럼? 어떤 것을 그대로 기억할 것인지를? 얻게 됨.

Sequence models

- Autoregressive models.



- → 다음 단어를 예측하는 방식.

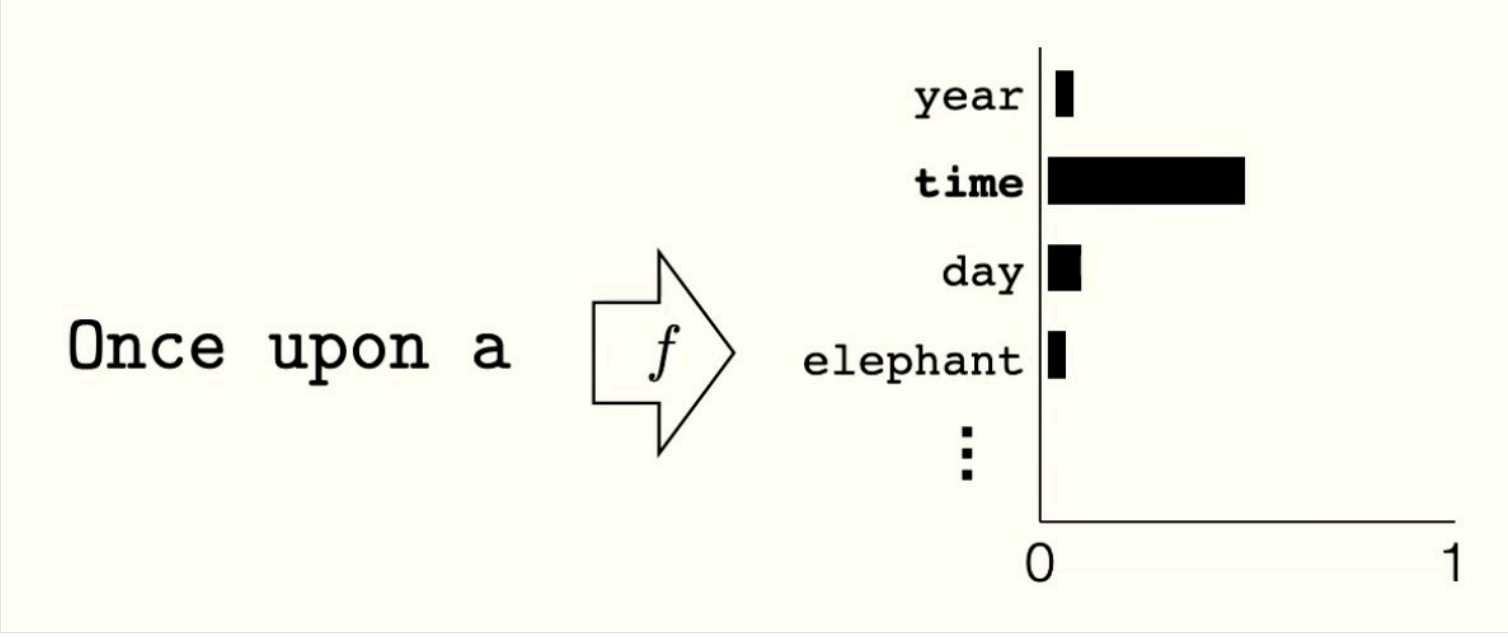


- Sequence model의 언어 모델 학습 방식에는 다음 토큰 예측(next-token prediction)과 마스킹된 토큰 예측(masked language modeling) 등이 있다. 두 방식은 서로 구분해야 한다.
- GPT류의 autoregressive language model은 이전 토큰들을 조건으로 다음 토큰을 예측하는 방식으로 학습한다. 각 위치에서 왼쪽 문맥만 참조하며, 미래 토큰을 직접 참조하지 않도록 causal masking을 적용한다.

- 반면 BERT류의 masked language model은 입력 시퀀스의 일부 토큰을 마스킹하고, 주변 문맥을 이용하여 해당 토큰을 예측하는 방식으로 학습한다. 이때 마스킹된 위치의 왼쪽과 오른쪽 문맥을 모두 활용할 수 있다.
- 따라서 다음 토큰 예측은 autoregressive 학습 방식이고, 빈칸을 채우는 masked language modeling은 양방향 문맥을 활용하는 별도의 학습 방식이다.
- 문장 또는 토큰 시퀀스 전체의 결합 확률(joint probability)은 확률의 연쇄 법칙(chain rule)을 이용하여 각 토큰의 조건부 확률의 곱으로 분해할 수 있다.
- 토큰 시퀀스를 $x_1, x_2, \cdots, x_T$라고 하면,
- $P(x_1, \cdots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_{1:t-1})$
- 여기서 오른쪽의 $x_{1:t-1}$는 현재 토큰 x_t보다 앞에 있는 모든 토큰을 의미한다. 첫 번째 토큰에는 이전 문맥이 없으므로 첫 번째 항은 $P(x_1)$이다.
- 이를 전개하면,
- $P(x_1, \cdots, x_T) = P(x_1) \times P(x_2 \mid x_1) \times P(x_3 \mid x_1, x_2) \times \cdots \times P(x_T \mid x_1, \cdots, x_{T-1})$
- 즉, 시퀀스 전체의 확률은 첫 번째 토큰의 확률과, 이후 각 토큰이 앞선 토큰들을 조건으로 등장할 확률을 모두 곱한 값이다.
- Autoregressive language model은 이러한 조건부 확률을 순차적으로 모델링하여 다음 토큰을 예측한다.

Modeling a sequence of words

- → 그냥 **word classifier** 문제로 볼 수 있음



- softmax를 이용하여 다음 토큰 후보들에 대한 확률분포를 계산함.
- Softmax 자체가 하나의 토큰을 고르는 것은 아니며, 실제 토큰 선택은 이 확률분포를 바탕으로 sampling하거나 argmax로 가장 확률이 높은 토큰을 선택하는 별도 단계에서 이루어짐.

- 근데 단어가 너무 많음 (K=100,000)
- 그래서 그냥, 단어 자체를 class로 두는 방법도 있음.
  → 훨씬 많이 예측해야 해서, 단어가 쌓일 수록 안 좋아질 수 있음.

- ⇒ 단어 전체(word)와 문자(character) 사이의 단위인 subword token(예: BPE로 만든 토큰)을 예측 단위로 사용하는 방법이 있음.
- 즉, byte 쌍 자체를 예측한다는 뜻이 아니라, BPE(Byte Pair Encoding) 등으로 구성한 subword 토큰 중 다음에 올 토큰을 예측하는 것.